Resource-Bounded CoinVault Knowledge Pipelines

A mathematical and software-architecture foundation from MySQL snapshot to safe RAG serving

Code, diary, and design analysis of CoinVault and RagKit

2026-08-12

Contents

Executive summary

CoinVault's recent memory work solved a real production failure. The original index build retained the normalized corpus, stripped corpus, chunks, two representation families, and every vector in Go memory while also constructing Bleve and SQLite indexes. At the measured full shape - 19,977 documents, 57,053 chunks, 114,106 representations, and 1,536-dimensional vectors - the raw vector payload alone is 701,067,264 bytes, or about 668.6 MiB, before slice headers, backing-array slack, strings, JSON buffers, maps, database pages, lexical-index buffers, provider/cache state, and the Go runtime. The full build was killed in ECS even with an 8 GiB task allocation.

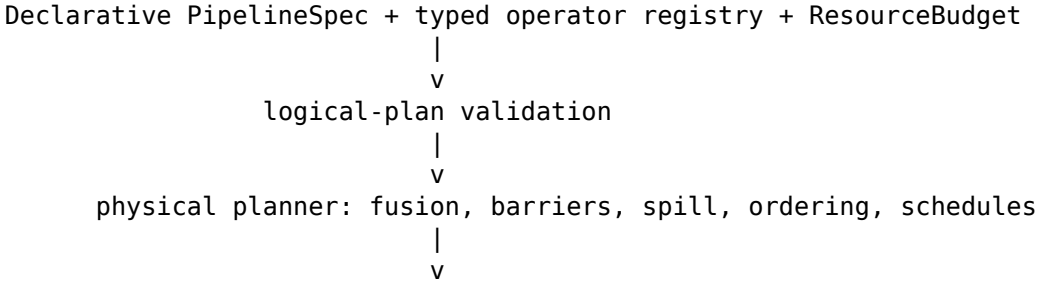
The remediation is valuable and should be retained. CoinVault now embeds representation blocks and stages vectors rather than accumulating one corpus-sized vector slice. RagKit's `BuildStream` writes ordered documents, chunks, representations, and vectors into a fail-closed SQLite staging relation; scans that relation to build a disk-backed content store, Bleve index, and SQLite vector index; seals semantic digests; and publishes by atomic rename. The verifier was made streaming, serving moved corpus text out of resident slices into `content.sqlite`, authorization occurs before hydration, and exact-vector scoring reads float components directly from SQLite blobs rather than allocating a decoded slice per row. Synthetic and local full-shape proofs show that these changes substantially reduce peak memory.

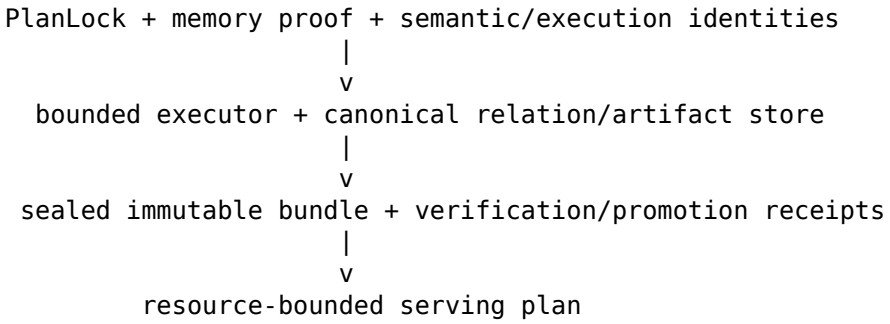
The concern that the work is still "disparate" is correct. The implementation is memory-bounded only from the **bundle-admission boundary onward**. The stages before that boundary still materialize global slices and maps:

- all product facets are loaded into a `map[productID][]facet`;
- all admitted product, category, and SQL-document records are collected into `[]rag.Document`;
- all documents are globally sorted;
- corpus JSON is encoded into one full `[]byte` before being written;
- furniture stripping copies the corpus and retains per-document token spans, complete shingle strings, and a global frequency map;
- all chunks are retained in `[]rag.Chunk`;
- raw and breadcrumb representations are built as full slices and composed into another full slice;
- only then are those already-materialized values copied into the bounded RagKit stager.

The resulting system therefore has two different execution models: an eager, product-specific front half and a streamed, storage-backed back half. It also has several overlapping durable forms: original corpus JSON, indexed corpus JSON, embedding cache files, temporary `staging.sqlite`, `chunks.json`, `representations.json`, `content.sqlite`, Bleve files, `vectors.sqlite`, bundle manifests, export archives, receipts, EFS generations, S3 diagnostics, and activation configuration. Each form has its own validation, telemetry, packaging, recovery, and operational command. The 2026-08-10 OOM effort consequently touched index construction, verification, serving, exports, diagnostics, profile resolution, ECS task sizing, EFS ownership, S3 transfer, deployment images, startup instrumentation, and query-time vector allocation. That breadth is not evidence of poor engineering; it is evidence that the missing abstraction is a **resource-bounded dataflow and artifact lifecycle**, not another CoinVault-specific helper.

The recommended foundation is a small **resource-bounded pipeline compiler and runtime** shared by CoinVault, RagKit, and eventually RagOpt:





The central simplification is one **canonical disk-backed relation store**. MySQL rows are read under a declared snapshot and normalized into ordered document records. Global algorithms, such as furniture detection, use bounded external-memory passes over that store. Chunking and representation construction run per document and immediately append their outputs. Embeddings are produced in byte-bounded batches and written to a transactional content-addressed cache and relation. Lexical, vector, and content indexes become deterministic projections over the same sealed relations. The temporary staging database should either become the canonical bundle database or be replaced by a generic relation-store interface; it should not be a short-lived duplicate of data then copied into several additional authoritative stores.

Spivak and Fong's categorical approach is useful here, but only at the **logical semantics and composition layer**. CoinVault's MySQL schema, canonical document schema, chunk/representation schema, and serving views can be treated as typed schemas connected by composable migrations. This gives a precise vocabulary for structural integration, view restriction, lineage, and equivalence laws. It does not provide a memory manager, an external-sort algorithm, or a safe provider runtime. The practical design is therefore two-level: categorical/schema semantics describe what a migration means; relational and external-memory physical operators describe how it runs under a budget. The implementation should borrow the laws and interfaces, not build a general-purpose Kan-extension engine in the production path.

Each operator must declare more than a Go function. It needs a machine-readable contract:

- input and output schemas;
- key, ordering, and partitioning requirements;
- streaming, barrier, windowed, or external-memory execution class;
- cardinality and byte-expansion estimates;
- maximum retained state, workspace, and in-flight bytes;
- spill/checkpoint strategy;
- deterministic identity and side effects;
- retry, idempotence, and provider-budget behavior;
- security labels and trust boundary.

The planner then admits a run only when it can prove a conservative phase bound:

$$M_p \leq M_{runtime} + M_{native,p} + \sum_{e \in E_p} C_e + \sum_{v \in V_p} (W_v + S_v + I_v) + M_{margin}$$

where `C_e` is the byte capacity of an active edge, `W_v` is operator workspace, `S_v` retained state, `I_v` its largest admitted batch, and `M_native,p` covers opaque native backends such as Bleve. Batch sizes must be byte-based, not merely record-count based. Memory permits should be hierarchical and released when batches leave scope, giving backpressure a concrete resource meaning.

For the current CoinVault path, the first implementation target should be deliberately narrow:

1. Open one read-only, consistent MySQL snapshot.
2. Stream products and product facets in product-ID order using a merge join; do not build a corpus-sized facet map.
3. Merge the already ordered product, category, and SQL-document cursors by document ID; do not globally sort a full slice.

4. Append normalized documents to a canonical SQLite relation in bounded transactions while producing a streaming corpus artifact.
5. For a no-furniture baseline, read one document family at a time, chunk it, create raw and breadcrumb representations, embed byte-bounded batches, and append all relations immediately.
6. Build content, lexical, and vector projections sequentially under explicit native-memory reservations.
7. Seal identities, deep-verify offline, atomically publish, and produce an activation plan.
8. Prove identical retrieval behavior and bounded memory on the full corpus under a selected hard limit.
9. Add furniture as a separate two-pass external aggregate, not as an in-memory corpus transform.

This report describes the current path end to end, formalizes its memory and I/O behavior, proposes the operator model, IR/DSL, plugin interfaces, physical algorithms, serving changes, engineering workflow, and migration plan. It is a companion to the earlier self-optimizing RAG foundation report: the same canonical plan and typed patch model can make resource policies, index backends, chunkers, and query plans optimizable without compromising memory proofs or evidence custody.

1. Scope, evidence, and limitations

1.1 Revisions and artifacts analyzed

The analysis uses the supplied CoinVault archive, its recovered OOM investigation material, and the current review branches referenced by that work:

- CoinVault archive: `gec-rag-opt.zip`, extracted under `/mnt/data/coinvault-analysis/coinvault`.
- CoinVault review branch: `goldeneagle/coinvault, agent/profile-backed-embeddings`, observed at commit `e63a5029454a84c7872767c46d00ee1002fa48b8`.
- RagKit review branch: `go-go-golems/ragkit`, `agent/oom-build-observer`, observed at commit `f4bef38d569757ea8af5bd5cd582e0e042ddca3a`.
- CoinVault's pinned RagKit pseudo-version: commit `5a4a5e1f38451d1e79a71f0d951b347d5a2c807b`.
- Ticket material under `ttmp/2026/08/10/COINVAULT-INDEX-OOM-001--bounded-memory-full-knowledge-bundle-build`.
- The earlier `self_optimizing_rag_foundation_report.md`, used only to keep the proposed plan/optimizer boundaries consistent.

The extracted archive has a broken Git worktree pointer and omits `internal/knowledgebuild`, even though the command imports it. The missing package was recoverable from investigation traces and from the private GitHub review branch. This is a reproducibility limitation and also a concrete ownership smell: an archive that appears to contain the application does not contain a build-critical package because worktree state and captured traces became part of the effective source of truth.

The feature work is substantial. Relative to the current main branches, the review branches are:

Repository	Commits ahead	Character of changes
CoinVault	28	About 1,000 added lines in the main knowledge command; memory/EMF telemetry; embedding profile resolution; cache and bundle exporters; seed/state inspection; schema-v2 serving integration; deployment tests and manifests

Repository	Commits ahead	Character of changes
RagKit	23	Streamed bundle builder and staging kernel; streaming verifier; disk content store; paged Bleve inspection; vector build/inspection changes; direct blob scoring; schema-v2 bundle changes

This count is not a criticism of the commits. It quantifies why the effort felt cross-cutting: one resource failure required changes across product semantics, generic indexing, serving, verification, observability, packaging, deployment, and recovery.

1.2 Static-analysis limitation

The supplied module requires Go 1.26.5. The available local toolchain is Go 1.23.2, so the repository tests could not be executed in this environment. The report relies on source inspection, ticket measurements, branch diffs, and existing test descriptions. It distinguishes:

- **observed behavior**: directly implied by source or recorded measurements;
- **inference**: a consequence derived from the source model;
- **proposal**: a recommended architecture or algorithm.

No claim in this report should be read as a fresh production benchmark.

1.3 What “memory-bounded” means here

A pipeline is not memory-bounded merely because it processes vectors in blocks. It is memory-bounded when, for a declared input envelope and execution plan, there is a conservative upper bound on resident memory that does not grow with total corpus size.

Let `N` be total records and `B` be the configured process memory budget. A stage is **asymptotically bounded** when its retained state is `O(1)`, `O(batch)`, `O(window)`, `O(partitions)`, or an explicitly bounded external-memory index, rather than `O(N)`. A complete pipeline is operationally bounded when:

1. every edge and operator has a byte bound;
2. every global operation has a spill/external algorithm or an explicit finite input ceiling;
3. native libraries and file cache receive reservations or safety margins;
4. the physical scheduler limits overlapping high-water stages;
5. cancellation and failure release reservations and temporary artifacts;
6. the plan is tested under a hard cgroup/container limit.

This is stricter than “streaming API” and more useful than peak-memory telemetry after the fact.

2. The current system, end to end

2.1 Control and configuration surface

A CoinVault knowledge generation is assembled from several independent control surfaces:

- a committed knowledge manifest describing sources, chunking, embeddings, and furniture stripping;
- MySQL connection and application profile settings;
- an embedding profile registry that resolves provider type, model, endpoint, dimensions, and credentials;

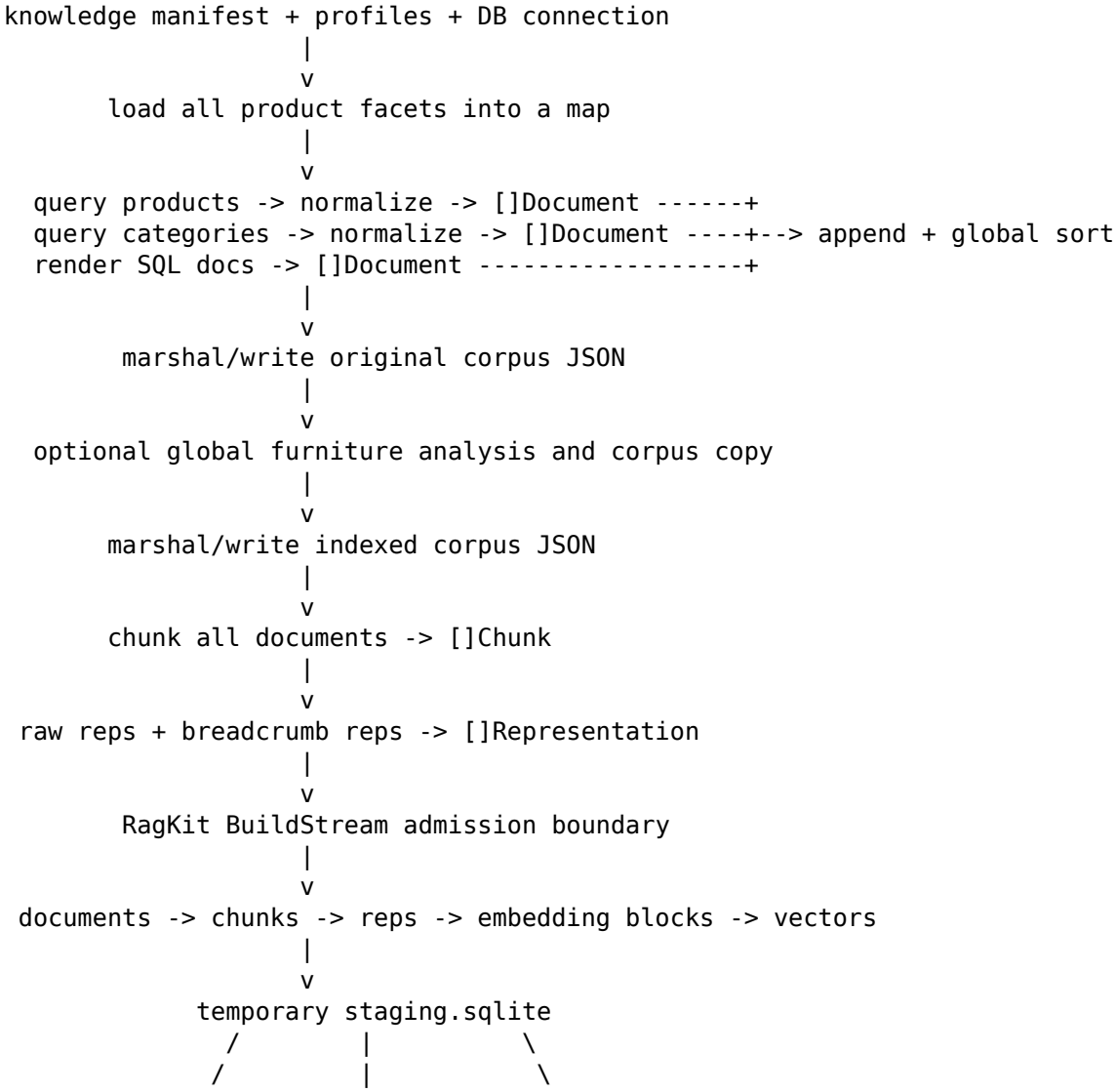
- CLI flags for output roots, manifests, bundle paths, profiles, telemetry, exports, verification, evaluation, sweeps, and judges;
- environment variables for serving-time reranker and synonym behavior;
- RagKit semantic identities and bundle schema versions;
- EFS paths, S3 destinations, task images, task CPU/memory, IAM roles, and schedules in Terraform/ECS;
- the active serving command's explicit `--knowledge-bundle` path;
- cache, export, seed, state-inspection, verify, eval, sweep, judge, and package subcommands.

The knowledge CLI has grown to approximately 2,433 lines. It is not just a command adapter: it is an operational control plane for build, diagnostics, recovery, publication, verification, evaluation, and provider accounting. This makes every new cross-cutting concern easy to add locally but hard to reason about globally.

A compiled pipeline lockfile should replace most of this implicit assembly. The authoring manifest may remain human-readable, but execution should consume one canonical object containing resolved source identities, operator versions, resource budgets, profile identities, artifact locations, and the physical plan.

2.2 Current offline path

The observed feature-branch path is:



```
      content.sqlite   Bleve      vectors.sqlite
              \         |          /
               \        |         /
      chunks.json + representations.json + manifest
                      |
                      v
          fsync + atomic directory rename
                      |
                      v
 verify / inspect / export / seed / publish / activate
```

The key boundary is the call to `indexbundle.BuildStream`. Everything below it uses bounded ordered scans. Everything above it is still corpus-eager.

2.3 MySQL product extraction

`LoadProductDocuments` first calls `loadProductFacets`, which executes:

```
SELECT product_id, name, value
FROM product_details
ORDER BY product_id, name, value
```

and retains every non-empty result in:

```
map[int64][]productFacet
```

The product query then scans active products ordered by product ID. It deliberately excludes price, cost, and quantity: those values remain live SQL facts rather than indexed prose. Each row is normalized by:

- converting HTML descriptions and short descriptions to text;
- rejecting descriptions below a rune threshold;
- selecting page title or product name;
- composing metal and EAV facets into a lexical summary;
- adding source, external ID, source role, access scope, timestamp, URL, metal, and facet metadata;
- hashing normalized text into a content digest.

This contract is sensible. The memory issue is the join algorithm. Both inputs are ordered by product ID, so a streaming merge join is available. The complete facet map is unnecessary.

A bounded implementation can hold only:

- the current product row;
- the current product's facets;
- one normalized document;
- a small output batch.

If SQL policy permits, server-side aggregation into deterministic JSON can also remove the client join, but an ordered merge join is easier to test, provider-independent, and preserves explicit normalization logic.

2.4 Category and curated SQL-document extraction

Categories are scanned in ID order, normalized, filtered by minimum description length, and returned as a complete document slice. Curated SQL documentation is generated by sorting topic and table names from an in-memory library and creating one document per topic/table.

Each source is individually ordered or can be made ordered. The current builder appends all sources and calls `sort.Slice(documents, ID)`. A bounded source union can instead perform a `k`-way merge over source cursors using a heap of size `k`. Here `k` is tiny: products, categories, and SQL docs. Memory becomes `O(k + batch)` rather than `O(N_documents)`.

2.5 Corpus serialization

The current build uses `json.MarshalIndent(documents, "", "  ")`, which creates a complete JSON byte slice before `os.WriteFile`. During serialization, the process can hold:

- all document structs and strings;
- the encoded JSON buffer;
- temporary encoder allocations;
- possibly the indexed-document copy if furniture is enabled.

This is a transient peak that stage-level sampling can miss. A streaming JSON array, JSON Lines file, or direct canonical relation write avoids the duplicate full-corpus buffer. For a bundle contract, JSON Lines is usually operationally superior because it supports append, line-level validation, bounded reads, and stable recovery. If compatibility requires an array, it can still be streamed with `[` / commas / `]`, as the new RagKit stager already does.

2.6 Furniture stripping

Furniture stripping is semantically global: a shingle is furniture when its document frequency exceeds a role-relative threshold, so the system cannot decide whether a span is furniture by looking at one document only.

The current implementation groups documents by source role, then retains for each role:

- token spans for every document;
- every shingle as a full Go string for every document;
- a global `map[string]int` document-frequency table;
- per-document coverage bitsets/runs;
- a copied output document slice;
- report structures.

For document `i` with `T_i` tokens and shingle width `q`, the number of shingles is approximately `T_i-q+1`. Storing each shingle as a string can approach the size of the corpus multiplied by overlap, plus string headers and hash-map overhead. This can dominate the supposedly expensive vector stage for verbose documents.

The correct abstraction is not "make `StripFurniture` accept a channel." It is a two-pass external aggregate:

1. **Frequency pass**: stream documents in `(role, documentID)` order; generate hashes of normalized shingles; deduplicate shingles within the current document; append `(role, shingleHash, documentID)` records or update a partitioned external count.
2. **Reduce pass**: external sort/group by `(role, shingleHash)` or use a disk key-value/SQL aggregate; compute threshold membership after role counts are known.
3. **Rewrite pass**: stream canonical documents again; tokenize one document; mark spans by lookup against the furniture set; emit a stripped document and audit events.

This bounds memory by one document, one per-document dedup set, a bounded sort/aggregation buffer, and lookup-cache pages. Exact shingle text can be retained only for selected report examples; the frequency relation can use collision-resistant hashes plus an optional exact-text guard for selected candidates.

2.7 Chunking and representations

The current builder calls `chunking.Apply` over all indexed documents and retains every chunk. It then creates raw representations for every chunk, creates breadcrumb representations using all documents and chunks, and composes both complete slices.

The structure of the operations is much more local than the implementation suggests:

```
one document
   -> heading-aware chunk sequence
```

```
-> for each chunk:
     raw representation
     breadcrumb representation using the current document headings
```

A document is already the natural partition. The chunker's overlap state is bounded by the current section/document. Breadcrumb construction only needs the current document and current chunk. There is no semantic need to retain prior documents, chunks, or representations once their records have been appended and their ordered digests updated.

This stage should be implemented as a partition-preserving transducer:

$$Document \to Chunk^* \to Representation^*$$

with an explicit maximum document size or a spillable document-text reader for pathological rows.

2.8 Embedding and cache

The feature branch improves the largest previous peak. Representations are embedded in blocks and each block is written immediately to the RagKit stager. A cached embedder uses a durable per-item file cache, with cache identity including the embedding task-prefix contract.

This is correct in three important ways:

- document-side and query-side prefix behavior is part of immutable vector-channel identity;
- provider/model/dimensions are checked against the resolved profile;
- a cache-only runtime rejects every miss, enabling provider-free recovery proof.

The remaining design issues are:

1. **Record-count batches**: a batch of 1,000 short representations and a batch of 1,000 very long representations have different memory and provider payloads. Limits should include bytes and tokens.
2. **File-per-item cache**: 113,719 cache files and 2.22 GB were observed. This is durable but creates inode, directory-scan, archive, upload, and reconciliation overhead. A transactional SQLite/LMDB/Badger-style cache or content-addressed packfiles would simplify export and state inspection.
3. **Cache and relation duplication**: vectors exist in cache entries, staging SQLite, and final vector SQLite. The system should define which store is authoritative and which copies are derivable.
4. **Provider admission**: workers and provider batches should consume call, token, byte, and memory permits from the same execution budget rather than being independent integers.

2.9 RagKit streamed bundle construction

`BuildStream` is the strongest new mechanism. It:

- validates output, chunker, embedding identity, producer, and batch size;
- creates a temporary bundle directory;
- opens a SQLite staging kernel;
- calls one producer with a strict phase order: documents, chunks, representations, vectors;
- validates each batch and writes it transactionally;
- seals corpus/chunk/representation/lexical/vector/content identities by ordered scans;
- writes chunk and representation JSON arrays by streaming from SQLite;
- builds `content.sqlite` from ordered producers;
- builds Bleve through ordered lexical records;
- builds the exact-vector SQLite index through ordered vector entries;
- verifies persisted identities;
- removes the staging database;
- writes the bundle manifest, fsyncs, and atomically renames the directory.

This achieves bounded memory for backend construction and preserves deterministic bundle identity. It also provides a useful fail-closed typestate: data cannot be read for index construction until the relation is sealed, and phases cannot be skipped or reordered.

Its limitations are architectural rather than correctness failures:

- the four phases are hard-coded into one package rather than described by a generic plan;
- batch limits are record counts, not byte/resource reservations;
- SQLite is used as a disposable interchange database and then its data is copied into multiple final stores;
- global ordering and semantic identity are specific to the bundle format;
- native-memory overlap is controlled by sequential source code rather than an explicit physical schedule;
- the caller can still materialize the entire input before entering the stager, which CoinVault does;
- the stager validates parent references through per-record database lookups, which can become an I/O bottleneck even when memory is bounded.

The right conclusion is not to discard `BuildStream`. It should become one backend or compatibility implementation behind a more general relation and execution contract.

2.10 Bundle contents and publication

A schema-v2 bundle includes at least:

- `manifest.json`;
- `chunks.json`;
- `representations.json`;
- `content.sqlite`;
- Bleve directory;
- `vectors.sqlite` when hybrid retrieval is enabled.

The output root also contains original/indexed corpus JSON and the embedding cache. Operational tooling can then package, export, inspect, verify, seed, or download generations. EFS is writable by the indexer and read-only to serving/inspection; exports use constrained S3 paths and receipts; activation is controlled by an explicit bundle path and service restart/deployment.

The immutable-generation and atomic-publication model is correct. The simplification opportunity is to collapse redundant representations and compile all publication actions from one artifact manifest.

2.11 Verification

The initial verifier eagerly loaded chunks and representations and reconstructed identities in memory. The revised verifier streams strict JSON arrays, keeps compact identity state, performs paged Bleve inspection, and preserves SQLite vector verification. This is an important improvement: verification must be resource-bounded too, or the system merely moves the OOM from build to startup or deployment gates.

Serving startup currently calls full `Verify` before opening the indexes. That gives strong fail-closed behavior, but it conflates two policies:

- **fast trust establishment**: verify manifest signature/digest, file sizes/hashes, schema versions, and backend embedded identities;
- **deep semantic audit**: scan every chunk, representation, lexical document, and vector to recompute logical digests.

Deep audit belongs in build/promotion and periodic integrity jobs. Normal process startup should perform an `O(number of files)` quick verification against a signed or trusted receipt, then open read-only stores. A configurable deep-startup mode can remain for high-assurance environments and diagnostics.

2.12 Serving startup and content hydration

The schema-v2 service no longer holds full document/chunk/representation slices. It opens:

- the manifest and verified bundle;
- a read-only SQLite content store;
- the Bleve lexical index;
- the exact-vector SQLite index, if present;
- a query embedder whose provider/model/dimensions and prefix channel match the bundle.

The content store supports bounded candidate-metadata, chunk, and document lookup. CoinVault uses it correctly:

1. retrieve representation hits;
2. collapse to chunks;
3. fetch only candidate metadata;
4. apply scope and source-role authorization;
5. hydrate text only for authorized candidates needed by reranking or final output.

This is a strong confidentiality and memory boundary. It should become an explicit operator sequence in a compiled serving plan rather than remain distributed across `service.go` and `content_lookup.go`.

2.13 Query execution

For each user query, the service:

1. resolves an optional reviewed comparison intent into one or more deterministic retrieval queries;
2. selects a default result limit of 5 when not supplied;
3. over-fetches each query to `limit * searchDepth`, with `searchDepth = 8`;
4. runs lexical retrieval, optional synonym expansion, representation-to-chunk collapse, and authorization;
5. runs vector retrieval, chunk collapse, and authorization when available;
6. fuses channels with weighted reciprocal-rank fusion;
7. optionally hydrates an authorized pool and calls a reranker;
8. blends reranker and fused orders with another RRF;
9. interleaves decomposed-query rankings, deduplicates by chunk;
10. re-authorizes canonical metadata and hydrates final document/chunk records;
11. writes bounded evidence-ledger entries and returns tool results.

The query path is memory-bounded because candidate pools are limited. It is not necessarily latency- or I/O-bounded. The exact-vector backend scans every vector for every vector query. At the measured shape:

$$114{,}106 \times 1{,}536 \times 4 = 701{,}067{,}264 \text{ bytes}$$

of raw vector payload are read/scored per query, excluding SQLite page and row overhead. That is 175,266,816 float components. Direct blob scoring reduces allocation to approximately `O(d + K)`, but time and data movement remain `O(Nd)`.

Exact scan is useful as a ground-truth backend, smoke/rollback backend, and small-corpus implementation. It should be one plugin, not the assumed production endpoint. A production plan should be able to choose HNSW, IVF, scalar/product quantization, a database-native vector index, or a remote service while keeping the same identity, authorization, fusion, evidence, and evaluation contracts.

3. What the recent work achieved

The current work should not be dismissed as over-engineering. It established several reusable invariants that the new foundation should absorb.

Problem	Implemented response	Architectural value to preserve
Full vector accumulation	Embed in blocks and stage immediately	Bounded producer/consumer path; provider-free replay
Eager backend construction	SQLite-backed `BuildStream` and ordered consumers	External-memory build; deterministic sealing; atomic publication
OOM uncertainty	Heap/RSS/cgroup/stage telemetry and EMF	Resource observations tied to semantic stages
Eager verifier	Streaming JSON validation and paged Bleve inspection	Verification as a bounded first-class workload
Resident serving corpus	`content.sqlite` and bounded lookups	Authorization-before-hydration; bounded startup/request memory
Per-row vector allocation	Cosine directly over SQLite blobs	Allocation-free exact scoring and rank parity
Interrupted provider work	Per-item embedding cache and cache-only runtime	Replayable side-effect boundary and fail-closed recovery
Opaque EFS state	State inspector, cache/bundle exporters, receipts	Artifact custody and remote diagnostics
Shared task sizing	Separate indexer and serving task capacities	Workload-specific resource envelopes
Accidental publication	Temporary directory, fsync, atomic rename	Immutable generation state machine
Unsafe activation	Explicit bundle path, read-only serving, rollback playbook	Separation of build, publish, and activation

Recorded evidence includes:

- 113,719 embedding cache files totaling about 2.22 GB;
- a provider-free reproduction reaching 56,000 cache hits before a fail-closed miss;
- about 1.36 GiB RSS near half-vector accumulation in the eager path;
- a synthetic `19,977 / 57,053 / 114,106 x 1,536` streamed build completing in 81.71 seconds with 270,336 KiB maximum RSS;
- an exact full rebuild with zero provider requests after cache reconciliation;
- a full rebuild peak of about 835,891,200 bytes, attributed near Bleve construction;
- serving startup/first-query peaks around 250 MiB under both 2 GiB and 4 GiB container limits.

These measurements show two different facts:

1. the structural vector peak was removed;
2. native lexical construction now dominates the measured full rebuild and therefore must be represented in the planner as an opaque or calibrated reservation.

The second fact is why a generic streaming API alone is insufficient. A correct physical plan must schedule native sinks and allocate headroom for their non-Go memory and file-cache behavior.

4. Why the work still feels disparate

4.1 The abstraction boundary is in the middle of the pipeline

The new RagKit builder is bounded, but CoinVault hands it fully materialized documents, chunks, and representations. The property “bounded” therefore does not compose from source to sink. The code contains bounded loops, but there is no end-to-end proof.

4.2 Logical semantics and physical execution are mixed

The same functions decide:

- what constitutes a document;
- how sources join;
- how records are ordered;
- what is materialized;
- where it is stored;
- how large batches are;
- how provider concurrency is controlled;
- how progress is logged;
- which backend is constructed;
- how artifacts are published.

Changing memory behavior risks changing semantic identity because the logical and physical plans are not separately represented.

4.3 Global algorithms are hidden behind collection APIs

`[]Document -> []Chunk`, `[]Document -> []Document`, and `[]Representation -> []Vector` make simple call sites but erase whether an operation is local, windowed, global, blocking, spillable, or side-effecting. The caller cannot plan memory because the interface does not expose the algorithmic class.

4.4 Artifact custody is implemented per incident

Cache export, bundle export, state inspection, seed download, receipts, verification, packaging, and activation are separate commands/packages because no generic artifact manifest describes:

- semantic role;
- producer plan;
- content digest;
- storage URI;
- size/counts;
- completeness state;
- verification level;
- promotion status;
- retention policy.

4.5 Observability is retrospective rather than prescriptive

Memory checkpoints answer “what happened?” A resource planner must first answer “can this plan be admitted under the declared limit?” Actual telemetry should then calibrate estimates and detect model violations.

4.6 The offline and online systems use different composition vocabularies

Offline composition is a product build function plus a fixed RagKit stager. Online composition is a large service with setters for fusion, reranking, synonyms, and comparison plans. The system lacks one typed graph vocabulary for both. This complicates optimization, lineage, and consistent security/resource policy.

4.7 Operational concerns leak into application commands

A 2,433-line knowledge command now understands S3 uploads, cache reproduction, EFS states, telemetry sinks, provider profiles, bundle exports, verification, evaluation, parameter sweeps, and judging. Those are legitimate capabilities, but they should be generated from reusable run/artifact interfaces rather than accumulated in one command family.

5. Theoretical foundations

5.1 Dataflow graph

Model a pipeline as a directed graph:

$$G = (V, E)$$

Each vertex is an operator and each edge carries typed records or batches. Operators have semantic properties and physical resource properties. The graph can include acyclic offline build plans and cyclic/interactive online plans, although the first implementation should keep offline execution acyclic.

A graph makes locality and barriers visible. For example:

```
MySQL products ----+
                   +--> merge join --> normalize --> canonical documents
MySQL facets -------+

canonical documents --> shingle map --> external group --> furniture set
canonical documents + furniture set --> strip --> chunk --> representations
representations --> embedding cache/provider --> embeddings
relations --> lexical sink
relations --> vector sink
relations --> content sink
```

5.2 Volcano-style iterators and push pipelines

The Volcano model treats operators as iterators with open/next/close and separates operators from the scheduler. Pure pull iteration gives natural backpressure and bounded state for many pipelines. Push-based batched execution is better for vectorized processing and providers. A practical kernel can support both through a common cursor/emitter boundary.

The important rule is that a batch is a **leased resource**, not just a slice. Its byte cost is reserved before creation and released after the downstream consumer finishes.

5.3 Synchronous dataflow and bounded buffers

Synchronous dataflow assigns known production/consumption rates to actors and can derive bounded schedules. RAG pipelines have data-dependent rates - one document creates a variable number of chunks and representations - so classic SDF is not directly sufficient. Its discipline is still useful:

- declare expected and maximum expansion rates;
- expose actor state and edge capacities;
- compute a schedule rather than spawning unconstrained goroutines;
- reject cycles without initial tokens or explicit state.

For variable-rate stages, the planner can use conservative upper bounds, runtime byte credits, and spill paths.

5.4 External-memory algorithms

When data exceeds memory, algorithm cost is dominated by block transfers rather than arithmetic. External sorting, partitioned hashing, merge joins, and sequential scans should be explicit physical operators. The Aggarwal-Vitter I/O model gives the conceptual basis: with memory `M`, block size `B`, and `N` records, external sorting requires roughly:

$$O\left(\frac{N}{B}\log_{M/B}\frac{N}{B}\right)$$

block transfers. The practical lesson is to prefer already ordered scans and merge joins, and to make every unavoidable global operation a planned external pass rather than an accidental map/sort over a full slice.

5.5 Relational algebra and ordered relations

The CoinVault build is naturally relational:

- products join facets;
- sources union into documents;
- documents expand into chunks;
- chunks expand into representations;
- representations left-join embeddings;
- indexes are materialized views.

Treating these as typed ordered relations enables familiar optimization rules:

- push filters before expensive transforms;
- use merge join when inputs share order;
- project only required columns;
- materialize at barriers/fanout, not every function boundary;
- use indexes for parent validation;
- preserve stable keys and ordering for deterministic digests.

This does not require exposing SQL to product developers. The IR can be relational while APIs remain typed Go.

5.6 Spivak/Fong functional data migration: useful, but not the execution engine

The functorial data-migration program gives a rigorous account of structural database transformation. In its basic form:

- a schema `S` is a finitely presented category;
- objects represent entity or value types;
- arrows represent attributes and foreign keys;
- path equations represent integrity constraints;
- an instance is a set-valued functor `I : S -> Set`;
- a schema functor `F : S -> T` induces an adjoint triple of data migrations:

$$\Sigma_F \dashv \Delta_F \dashv \Pi_F.$$

`Delta_F` is restriction by precomposition; `Sigma_F` is a left-Kan-extension style migration associated with integration, union, quotienting, and key generation; `Pi_F` is a right-Kan-extension style migration associated with product- and join-like completion. Spivak and Wisnesky show that a practical functorial query language is closed under composition and is relationally implementable using select, project, product, union, and key generation. The later implementation paper makes an important engineering point: direct select/from/where syntax can be executed more efficiently than mechanically expanding every query into a `Sigma-Pi-Delta` expression because ordinary join reordering and relational optimization remain available.

Where it maps cleanly onto CoinVault

CoinVault already moves through a sequence of implicit schemas:

```
S_mysql
  Product, ProductDetail, Category, SQLDoc, scalar types
      |
      v
S_document
  Document, SourceRef, Role, Scope, Metadata, Text
      |
      v
S_chunk
  Document <- Chunk, offsets, heading path
      |
      v
S_representation
  Chunk <- Representation, representation kind, content digest
      |
      v
S_embedding
  Representation <- Embedding, model identity, dimensions
      |
      v
S_bundle / S_serving
  content view, lexical view, vector view, evidence/tool view
```

Treating these as explicit versioned schemas is useful for several reasons.

1. **Structural integration becomes declared.** Product, category, and curated SQL-document sources map into one canonical `Document` schema instead of disappearing inside ad hoc constructors and slices.
2. **Lineage becomes compositional.** The path from a MySQL key to a document, chunk, representation, embedding, index entry, and final evidence item is a composition of named migrations and value transforms.
3. **Schema changes gain laws.** A schema-v2-to-schema-v3 migration can be checked against the direct source-to-v3 path. The intended square should commute, up to an explicitly declared identity mapping.
4. **Views become first-class.** Content, lexical, vector, evaluator, and tool projections are named views over the same sealed instance rather than unrelated file formats with loosely synchronized counts.
5. **Plugin compatibility becomes more precise.** A component can state the schema it consumes, the schema it produces, and the structural map it denotes. Compatibility is then more than matching Go interfaces.

A useful CoinVault reading of the three migrations is:

Migration idea	CoinVault use	Physical implementation
`Delta` restriction	expose a fixed serving/evaluation/index view from a richer bundle schema; forget fields not needed by an adapter	projection, filtered cursor, read-only view
`Sigma` integration	map products, categories, and SQL docs into a common document family; generate deterministic canonical IDs; union source families	ordered merge-union, canonical key function, append relation
`Pi` completion/join	assemble a product with its related facets or derive relation-valued context required by normalization	merge join, grouped cursor, indexed nested-loop for bounded cases

This table is an analogy at the design level, not a claim that every current function is literally one primitive migration. CoinVault normalization also cleans HTML, renders Markdown, assigns roles, excludes live facts, computes hashes, strips furniture, chunks text, and calls embedding providers. These are **value-level transformations and effects**, not merely changes of schema. They belong in typed operators whose structural input/output schemas are categorical, while their value semantics are ordinary code or a smaller algebra.

Wiring diagrams and a compositional kernel

Fong and Spivak's broader compositional viewpoint is also useful for the pipeline IR. Regard typed relation interfaces as objects and operators as morphisms. Sequential wiring is composition. Independent branches use a monoidal product. Fanout, materialization, and feedback require explicit structure rather than implicit shared slices.

The implementation need not expose category-theory terms. It should preserve the corresponding laws:

- composition is associative;
- identities are explicit no-op migrations;
- parallel composition is independent unless a shared resource/effect is declared;
- structural migrations compose independently of their physical algorithms;
- equivalent physical plans have the same denotation on finite instances;
- source-to-target diagrams required by compatibility policy commute.

This supports a two-level compiler:

```
Schema graph + migration/value-transform denotations
                        |
                        v
            logical relational/dataflow IR
                        |
                        v
   merge joins / external sort / partition aggregate / sinks
                        |
                        v
       resource-scheduled, checkpointed physical plan
```

A physical operator should therefore carry both:

```
Denotation:   the relation/schema transformation it promises
Realization:  the bounded algorithm, ordering, spill, and effect contract
```

For small generated instances, property tests can compare the external-memory realization with a simple eager reference evaluator. For migration composition, the compiler can check laws such as:

$$\Delta_{G \circ F} = \Delta_F \circ \Delta_G,$$

and the corresponding `Sigma` and `Pi` composition laws up to canonical isomorphism. In practical tests, this becomes equality of canonical relation rows and digests, not theorem-prover integration.

Resource semantics can also be compositional

Category theory does not supply CoinVault's memory bound, but the same compositional discipline helps define one. After the physical schedule is fixed, interpret every operator and edge in an ordered resource algebra. For example:

- serial materialized composition has peak approximately `max(M_f, M_g, edge/workspace)`;
- overlapped pipelined composition has peak bounded by `M_f + C_edge + M_g` plus shared/native overhead;
- parallel composition adds active reservations;
- sequential latency adds, while parallel critical-path latency takes a maximum;
- provider calls, bytes written, and temporary-disk use have their own additive or peak operations.

This interpretation is best viewed as a monotone, lax compositional map from a **physical** plan to a vector of resource bounds. It is deliberately schedule-aware: a logical wiring diagram alone cannot decide whether two actors overlap.

Where the approach does not fit directly

Functorial data migration should not be sold as the solution to the OOM problem.

- It does not choose batch sizes, queue capacities, or spill partitions.
- It does not model Go heap representation, Bleve's native/cache envelope, SQLite page behavior, provider limits, or cgroup pressure.
- Basic set-valued instances abstract away ordering, bag semantics, nulls, floating-point behavior, timestamps, and side effects unless these are modeled explicitly.
- Furniture detection is a corpus-level statistical aggregate; chunking is a value-expanding text algorithm; embedding is a paid nondeterministic external effect; approximate-vector construction is an algorithmic materialization. None is reduced to a schema functor alone.
- A literal generic Kan-extension executor would add theory and runtime complexity while discarding decades of relational and external-memory optimization.

Recommendation

Use Spivak/Fong as the **denotational chassis**:

- version schemas and schema maps;
- make structural migrations explicit and composable;
- record path equations and compatibility laws;
- use commuting-diagram/property tests for migration and view parity;
- let component descriptors expose their denotation;
- derive lineage and view identities from composed mappings.

Use relational algebra, Volcano-style cursors, external-memory algorithms, build-system identities, and explicit resource contracts as the **operational chassis**. The user-facing DSL should remain a practical pipeline/query language. Categorical notation may be emitted by `explain` or used in internal validation, but should not be required to author an ordinary CoinVault build.

5.7 Build systems and Merkle DAGs

A knowledge bundle is also a build-system result. `Build Systems a la Carte` separates three concerns:

- task description/dependencies;
- rebuilding strategy;
- execution.

CoinVault currently recomputes or revalidates large stages because dependencies and semantic keys are not represented as a reusable build graph. A Merkle-style artifact identity can make each relation and projection addressable by the digest of:

```
operator semantic version
+ canonical configuration
+ ordered input artifact identities
+ relevant external source snapshot identity
```

Execution policy - workers, batch bytes, temporary directory, retry timing - must be separate unless it changes semantics.

5.8 Monoids, folds, and deterministic identity

Counts, byte totals, min/max timestamps, cache statistics, and many digests can be computed as streaming folds. An associative combine operation enables partitioned execution and bounded aggregation.

Ordered semantic digests require additional care. Either:

- preserve canonical source order through every operator; or
- externally sort by stable key before sealing.

A digest should never depend on goroutine completion order.

5.9 Queueing and Little's law

Little's law states:

$$L = \lambda W$$

where `L` is average in-flight work, `lambda` throughput, and `W` residence time. Increasing workers or queue depth increases in-flight records and therefore memory. Provider concurrency should be selected from latency/throughput targets and resource budgets, not treated as a free speed knob.

The runtime should record:

- bytes and records admitted;
- residence time per edge;
- operator service time;
- blocked time waiting for memory/provider/disk permits;
- spill volume;
- queue high-water marks.

These observations calibrate future plans.

5.10 Typestate and effect systems

The staging kernel's phase checks are a local typestate. Generalize this into explicit lifecycle states:

```
planned -> admitted -> running -> sealed -> verified -> published -> activated
                       |                       |
                       +-> failed              +-> rejected
```

Operators also declare effects:

- pure deterministic transform;
- reads external snapshot;
- writes temporary artifact;
- calls paid provider;
- publishes immutable artifact;
- changes active deployment.

The compiler can then require stronger review and idempotence contracts as effects increase.

5.11 Separation of logical and physical plans

The logical plan says **what** relation and identity should be produced. The physical plan says **how** to produce it under a resource envelope.

Example logical operator:

```
furniture_strip(role, shingle_tokens=12, min_fraction=0.05, min_span=20)
```

Possible physical plans:

- in-memory frequency map for a proven small corpus;
- SQLite aggregate for medium corpus;
- partitioned hash + external merge for large corpus;
- reuse a previously sealed furniture relation when source digest is unchanged.

All physical alternatives must produce the same semantic output identity. This is the foundation for performance optimization without changing retrieval behavior.

6. Proposed architecture: a resource-bounded pipeline kernel

6.1 Six layers

The recommended implementation separates six layers that are currently interleaved.

1. Domain record model

Stable, versioned types for source rows, documents, chunks, representations, embeddings, candidates, evidence, and artifact identities. RagKit already supplies much of this layer.

2. Logical plan and operator registry

A serializable graph of typed operator instances. The registry resolves names and versions to descriptors and implementations. It validates schemas, keys, ordering, effects, security labels, and semantic configuration.

3. Physical planner

The planner chooses:

- pull, push, or batch execution;
- fusion of adjacent stateless transforms;
- edge byte capacities;
- in-memory versus external operators;
- partitioning and merge strategy;
- materialization/checkpoint boundaries;
- native backend sequence;
- resource reservations and provider concurrency;

- recovery points.

It emits a deterministic `PlanLock` and an explainable resource proof.

4. Bounded executor

The executor implements leased batches, hierarchical memory credits, cancellation, checkpoints, spill files, provider budgets, and event emission. It knows nothing about CoinVault document semantics.

5. Relation/artifact store

A local implementation can use SQLite plus immutable files. The interfaces support:

- ordered typed relations;
- transactional append and seal;
- cursor replay;
- semantic digests and counts;
- artifact manifests and locations;
- temporary, sealed, verified, published, and active states.

Remote object stores and warehouse backends can be added later without changing operator contracts.

6. Product adapters and operations

CoinVault supplies MySQL queries, normalization, role/scope policy, furniture configuration, chunker choice, embedding profile resolution, retrieval configuration, tool projection, and deployment activation. CLI commands become thin front ends over generic `plan`, `run`, `verify`, `publish`, and `activate` APIs.

6.2 One vocabulary for offline and online plans

The kernel should represent both build and serving graphs, while recognizing their different lifetimes.

```
OFFLINE PLAN

source.snapshot
  -> source.scan
  -> normalize
  -> global transforms
  -> chunk
  -> represent
  -> embed
  -> index projections
  -> seal/verify/publish
```

```
ONLINE PLAN

request.input
  -> query.normalize
  -> query.decompose
  -> lexical.search
   -> vector.search
   -> authorize.metadata
   -> fuse
   -> rerank
    -> authorize.final
   -> hydrate
   -> evidence/tool output
```

Shared concepts include:

- typed edges;
- component identity;
- resource budgets;
- authorization labels;
- traces and metrics;
- content-addressed artifacts;
- deterministic execution where possible.

Offline edges usually carry durable relations and can replay. Online edges usually carry request-scoped bounded batches and strict latency budgets.

6.3 Logical-plan IR

A compact first version can use these entities:

```
type Plan struct {
    APIVersion string
    Name       string
    Inputs     []InputBinding
    Nodes      []Node
    Outputs    []OutputBinding
    Policies   Policies
}

type Node struct {
    ID        string
    Component ComponentRef
    Config    json.RawMessage
    Inputs    map[string]PortRef
}

type ComponentRef struct {
    Kind    string // source, transform, aggregate, sink, service
    Name    string
    Version string
}

type PortSchema struct {
    Type        SchemaRef
    Key         []string
    Ordering    Ordering
    Partition   Partitioning
    Security    SecurityLabel
}
```

The IR should be canonical JSON or CBOR after compilation. Human-authored YAML/CUE is not itself the identity. Unknown fields are rejected. Defaults are expanded before hashing.

Schema-map denotations

The logical plan should optionally register versioned schemas and structural migrations:

```
type SchemaDef struct {
    Ref         SchemaRef
    Objects     []ObjectDef
    Arrows      []ArrowDef
    Equations   []PathEquation
}

type MigrationDef struct {
    Ref          MigrationRef
    Source       SchemaRef
    Target       SchemaRef
    Mapping      StructuralMapping
    ValueProgram *ComponentRef
```

```
    Laws          []LawRef
}
```

`StructuralMapping` is deliberately smaller than a general programming language. It identifies object/arrow mappings and key policy. `ValueProgram` names reviewed code for normalization, text processing, or other value-level behavior. The compiler lowers the combination into relational/dataflow operators. This preserves the useful functorial separation without requiring the executor to evaluate arbitrary categorical constructions.

6.4 Operator descriptor

Each registry entry describes semantics and physical capabilities:

```
type Descriptor struct {
    Ref          ComponentRef
    Inputs       map[string]PortSchema
    Outputs      map[string]PortSchema
    ConfigSchema SchemaRef
    Semantics    SemanticSpec
    Resources    ResourceSpec
    Effects      EffectSpec
    Capabilities CapabilitySet
}

type SemanticSpec struct {
    Deterministic bool
    OrderSensitive bool
    IdentityFields []JSONPointer
    CompatiblePhysicalVersions []string
}

type ResourceSpec struct {
    Class              ExecutionClass
    MinWorkspaceBytes  int64
    MaxWorkspaceBytes  int64
    MaxRecordBytes     int64
    MaxBatchBytes      int64
    RetainedState      StateBound
    Expansion          ExpansionBound
    Spill              SpillCapability
    NativeReservation  ReservationModel
}
```

A descriptor is inspectable without constructing the implementation. This is essential for planning, linting, DSL tooling, and optimization.

6.5 Execution classes

Use a small explicit taxonomy:

Class	Definition	CoinVault examples
`stream`	Output can be emitted after bounded input state	HTML-to-text, normalization, raw representation
`partitioned`	Bounded within a stable partition	heading chunking per document, breadcrumb representation

Class	Definition	CoinVault examples
`windowed`	Retains an explicit bounded window	overlap chunking, small query history
`barrier`	Requires all logical input before output	final identity seal, threshold decision
`external`	Global operation with disk spill	sort, shingle frequency, dedup, large join
`sink`	Materializes a relation or backend	SQLite content, Bleve, vector index
`side_effect`	Calls external provider or deployment API	embedding, reranker, S3 publish, activation

An operator that declares `barrier` without a finite state bound or spill strategy should be rejected for an unbounded corpus.

6.6 Physical-plan IR

The physical plan should make memory and materialization explicit:

```
type PhysicalPlan struct {
    LogicalDigest string
    Budget        ResourceBudget
    Stages        []Stage
    Edges         []PhysicalEdge
    Estimates     PlanEstimate
}

type Stage struct {
    ID           string
    Operators    []PhysicalOperator
    Parallelism  int
    Reservation  Reservation
    Checkpoint   *CheckpointSpec
}

type PhysicalEdge struct {
    From, To     PortRef
    BufferBytes  int64
    Materialized *RelationRef
}
```

A stage is a set of operators permitted to overlap. Native sinks such as Bleve can receive a calibrated exclusive reservation and run sequentially with other heavy sinks.

6.7 Compiler passes

A useful compiler pipeline is:

1. **Parse and default** - strict schema, explicit defaults.
2. **Name/type resolution** - resolve component versions and port schemas.
3. **Graph validation** - no missing ports, illegal cycles, duplicate IDs, or unreachable outputs.
4. **Ordering analysis** - prove or insert order-preserving merge/sort operators.
5. **Security-flow analysis** - prevent higher-scope text from entering lower-trust sinks or external services.
6. **Effect analysis** - locate provider calls, publication, and activation.
7. **Cardinality/size estimation** - combine source statistics and expansion models.

8. **Materialization planning** - insert relation boundaries at fanout, barriers, expensive side effects, and recovery points.
9. **Resource scheduling** - size buffers, batches, workers, and native reservations.
10. **Identity compilation** - semantic hashes, execution hash, input locks, output expectations.
11. **Policy checks** - hard memory/disk/provider limits and required verification.
12. **Emit `PlanLock` and `explain` report**.

The compiler should be deterministic: the same resolved registry and input spec produce byte-identical lockfiles.

7. Resource algebra and admission control

7.1 Memory terms

For an active physical stage `p`, define:

- `M_runtime`: Go runtime, static process state, profiles, logging, and base libraries;
- `M_native,p`: native/backend reservation, including C allocations and expected page/file cache not visible in Go heap;
- `C_e`: byte capacity of each active edge buffer;
- `W_v`: operator workspace;
- `S_v`: retained state;
- `I_v`: maximum in-flight input/output batch memory;
- `M_margin`: uncertainty and fragmentation reserve.

Admission requires:

$$\max_p M_p \le B$$

with:

$$M_p = M_{runtime} + M_{native,p} + M_{margin} + \sum_{e \in E_p} C_e + \sum_{v \in V_p} (W_v + S_v + I_v)$$

This is a conservative accounting identity, not a claim that all terms are statistically independent.

The bound is compositional only after scheduling decisions are explicit. For physical plans `f` and `g`, useful conservative rules are:

$$M(f ;_{barrier} g) \le \max(M(f), M(g), C_{materialized}) + M_{shared},$$

$$M(f \rhd g) \le M(f) + C_{edge} + M(g) + M_{shared},$$

$$M(f \otimes g) \le M(f) + M(g) + M_{shared}.$$

Here `;_barrier` is serial execution separated by durable materialization, `triangleright` is an overlapped pipeline, and `otimes` is parallel composition. The planner, not the logical schema layer, chooses among these realizations. This is the concrete resource interpretation of the compositional model described in Section 5.6.

7.2 Hierarchical byte permits

Implement a root memory pool for the run and child pools for stages/operators. A batch reserves bytes before allocation:

```
type Permit interface {
    Bytes() int64
    Release()
}

type MemoryPool interface {
    Acquire(ctx context.Context, bytes int64) (Permit, error)
    TryAcquire(bytes int64) (Permit, bool)
    Child(limit int64) MemoryPool
}
```

A `Batch[T]` owns a permit:

```
type Batch[T any] struct {
    Items  []T
    Bytes  int64
    permit Permit
}

func (b *Batch[T]) Release() {
    if b.permit != nil {
        b.permit.Release()
        b.permit = nil
    }
}
```

Operators may not retain an item after the batch is released unless they acquire a separate retained-state permit or copy it into a durable relation.

7.3 Estimating bytes

`unsafe.Sizeof` is insufficient because strings, slices, maps, and provider payloads reference external backing storage. Every schema should have a conservative encoded-size estimator, preferably measured from canonical wire bytes plus known object overhead.

Use at least:

- `record.CanonicalBytes()` for durable/wire size;
- `record.EstimatedResidentBytes()` for Go/native residency;
- provider token/byte estimates for remote calls;
- observed correction factors per operator implementation.

The planner should take the larger of a static estimate and a high percentile from prior runs, then add margin.

7.4 Record-size limits

A total-memory budget cannot save a process from one pathological row. Sources and operators need explicit limits:

- maximum MySQL row/document bytes;
- maximum HTML and normalized text bytes;
- maximum metadata size and key count;
- maximum chunk bytes/runes;
- maximum representation bytes/tokens;

- maximum provider batch bytes/tokens;
- maximum vector dimensions;
- maximum response/evidence bytes.

Records exceeding the limit need a declared policy: reject, quarantine, truncate with a semantic marker, or spill to a streaming large-object path. Silent truncation is not acceptable.

7.5 Expansion bounds

For operator v, describe expected and hard expansion:

$$N_{out} \leq \alpha_v N_{in} + \gamma_v$$

and byte expansion:

$$D_{out} \leq \beta_v D_{in} + \delta_v$$

Examples:

- source normalization: approximately one document per admitted row;
- heading chunking: bounded by document length and overlap;
- two representations per chunk: exact factor 2 for current raw+breadcrumb configuration;
- embedding: one fixed-dimensional vector per representation.

The compiler can propagate these bounds and warn when a configuration creates explosive overlap or representation fanout.

7.6 Native reservations

Some backends do not expose exact memory use. Treat them as calibrated black boxes:

```yaml
native_reservation:
  kind: empirical-envelope
  fixed_bytes: 128MiB
  per_record_bytes: 2048
  percentile: p99
  minimum_samples: 5
  safety_factor: 1.35
```

Bleve construction, SQLite cache/page behavior, compression, and cgo allocations belong here. The measured full build suggests Bleve is now the dominant peak, so the first planner should schedule Bleve separately and reserve from historical high-water evidence.

7.7 Disk and I/O budgets

Memory boundedness can move failure to disk. Plan admission must also estimate:

$$D_{temp} + D_{outputs} + D_{spill} + D_{cache-growth} \leq D_{available} - D_{reserve}$$

Track sequential bytes read/written, random IOPS, spill volume, and temporary amplification. The current design can hold several copies of the same semantic data; a relation-store consolidation should reduce disk amplification as well as complexity.

7.8 Provider budgets

Embedding is a side-effecting operator with multiple limits:

- concurrent requests;
- requests per minute;
- tokens/bytes per request and minute;
- total calls/tokens/cost per run;
- retry budget;
- uncertain usage after timeout.

The resource runtime should issue a compound reservation before a call. A request proceeds only after acquiring memory, rate, call, token, and cost permits. On ambiguous timeout, the conservative policy charges the reservation until reconciled.

7.9 Scheduling

Do not equate a graph with “one goroutine per node.” A bounded scheduler can:

- fuse adjacent stateless operators into one loop;
- run source and normalization concurrently with a small leased buffer;
- materialize before a global pass;
- serialize native sinks;
- allocate more workers only while memory and provider permits remain;
- pause producers when spill or downstream write latency increases.

For the initial CoinVault plan, simple sequential stages with bounded intra-stage concurrency are preferable to a highly concurrent DAG runtime.

7.10 Runtime enforcement and model violation

The memory proof is conservative but must be checked. The executor should sample heap, RSS, cgroup, and native stage counters. If actual use crosses a warning threshold:

1. stop increasing concurrency;
2. shrink adaptive batches;
3. force spill where supported;
4. abort before the cgroup hard limit if the plan cannot recover;
5. mark the resource model violated and save evidence.

A successful run that exceeded its declared plan bound is not a clean success; it is a calibration failure requiring review.

8. The canonical relation store

8.1 Purpose

The canonical relation store is the one intentional durable boundary between source ingestion and derived projections. It provides:

- bounded append;
- ordered replay;
- stable keys and parent relations;
- exact counts and semantic digests;
- resumable checkpoints;
- queryable diagnostics;
- a basis for build reuse;
- a direct content-serving implementation when appropriate.

SQLite is a pragmatic first backend. The interface must not expose SQLite-specific SQL as the semantic contract.

8.2 Proposed relations

A first schema can contain:

```
source_snapshot(
  snapshot_id TEXT PRIMARY KEY,
  source_kind TEXT,
  consistency_token TEXT,
  opened_at TEXT,
  config_digest TEXT
);

document(
  id TEXT PRIMARY KEY,
  ordinal INTEGER UNIQUE,
  source_uri TEXT,
  title TEXT,
  text TEXT,
  content_digest TEXT,
  metadata_json BLOB,
  source_snapshot_id TEXT,
  source_role TEXT,
  access_scopes TEXT
) WITHOUT ROWID;

exclusion(
  source_id TEXT,
  external_id TEXT,
  reason_code TEXT,
  details_json BLOB
);

furniture_shingle(
  source_role TEXT,
  shingle_hash BLOB,
  document_frequency INTEGER,
  PRIMARY KEY(source_role, shingle_hash)
) WITHOUT ROWID;

indexed_document(
  id TEXT PRIMARY KEY,
  text TEXT,
  content_digest TEXT,
  transformation_digest TEXT
) WITHOUT ROWID;

chunk(
  id TEXT PRIMARY KEY,
  ordinal INTEGER UNIQUE,
  document_id TEXT,
  document_ordinal INTEGER,
  byte_start INTEGER,
  byte_end INTEGER,
  text TEXT,
```

```
  content_digest TEXT,
  chunker_id TEXT,
  UNIQUE(document_id, document_ordinal)
) WITHOUT ROWID;

representation(
  id TEXT PRIMARY KEY,
  ordinal INTEGER UNIQUE,
  chunk_id TEXT,
  kind TEXT,
  text TEXT,
  content_digest TEXT,
  representation_id TEXT
) WITHOUT ROWID;

embedding(
  representation_id TEXT PRIMARY KEY,
  embedding_contract_digest TEXT,
  dimensions INTEGER,
  values_blob BLOB,
  cache_status TEXT
) WITHOUT ROWID;

relation_identity(
  relation_name TEXT PRIMARY KEY,
  schema_version TEXT,
  record_count INTEGER,
  semantic_digest TEXT,
  sealed_at TEXT
) WITHOUT ROWID;
```

The schema is illustrative. It should avoid storing identical large text in multiple tables when a relation can reference canonical content or a compressed blob store.

8.3 Original versus indexed text

CoinVault correctly preserves original source text for review and uses stripped text for indexing. Represent this as explicit lineage rather than two top-level JSON files:

```
source document --[normalize]--> canonical document
canonical document --[furniture-strip:v1]--> indexed document
```

Both carry digests and transformation identity. A review export can render original and indexed JSON from the relation store when needed. The relation store is the authority; JSON is an artifact view.

8.4 Staging database versus final content database

The current `staging.sqlite` already contains documents, chunks, representations, and vectors. It is then scanned into `content.sqlite` and `vectors.sqlite`, while chunk/representation JSON is also emitted. There are three reasonable designs:

Option A: promote staging into the canonical bundle database

After sealing, create read-only indexes/views and rename it as `bundle.sqlite`. Serving reads documents/chunks; vector backend can read the same embedding table; build diagnostics remain available. Bleve remains a separate projection.

Advantages: one copy, one identity boundary, simplest lineage.

Risks: write-optimized staging schema and read-optimized serving schema may conflict; schema migrations need care; exact vector scans and content queries share one database/file cache.

Option B: canonical build relation plus specialized final stores

Retain a sealed `relations.sqlite` as lineage authority. Build optimized `content.sqlite`, vector/ANN files, and Bleve as derived projections. Optional JSON views are generated only when requested.

Advantages: clear authority and rebuildability; backends can optimize independently.

Risks: more disk; publication bundle is larger.

Option C: discard canonical relations after projection

This resembles the current design, but the run store keeps a compact relation manifest and source snapshot.

Advantages: smallest final bundle.

Risks: harder diagnosis, partial rebuild, and proof; redundant JSON may still be needed.

The recommended first implementation is **Option B** during migration, because it preserves inspectability and lets existing bundle APIs coexist. After usage is understood, production publication can choose whether to retain relations or archive them separately.

8.5 Transaction and checkpoint model

Every relation append uses bounded transactions and durable checkpoints:

```
checkpoint = {
  operator_id,
  input_artifact_digest,
  last_committed_key,
  record_count,
  rolling_digest_state,
  output_relation_generation
}
```

Resume verifies the plan lock, input identity, relation schema, last key, and digest state before continuing. Provider results are committed atomically with cache entries and output relation rows so a crash does not create paid-but-unreferenced work when avoidable.

8.6 Semantic versus physical identity

Keep at least three identities:

1. **Semantic relation identity**: canonical records and their order.
2. **Physical artifact identity**: actual SQLite/Bleve/archive bytes.
3. **Execution identity**: physical plan, resource policy, implementation versions, and environment.

Two physical implementations may share the same semantic identity. This lets a memory-optimized plan prove behavioral parity with the eager plan without requiring byte-identical database pages.

9. Physical algorithms for the CoinVault build

9.1 Consistent MySQL snapshot

A deterministic build should not independently query facets, products, and categories while concurrent writes can produce a mixed-time corpus. The source operator should declare and record snapshot semantics:

- repeatable-read transaction or a database-supported consistent snapshot;
- database/cluster identity;
- transaction isolation;
- source query versions/digests;
- start timestamp and, where available, GTID/binlog/LSN token;
- source row counts and max update timestamps for diagnostics.

The source snapshot identity is not the corpus identity, but it is essential custody evidence.

9.2 Product/facet merge join

Both product and facet scans are ordered by product ID. Use a merge cursor:

```
facet := nextFacet()
for product := nextProduct(); product != EOF; product = nextProduct() {
    facets = empty bounded slice
    while facet.product_id < product.id:
        report orphan/unmatched facet according to policy
        facet = nextFacet()
    while facet.product_id == product.id:
        append facet subject to per-product byte/count limit
        facet = nextFacet()
    emit normalize(product, facets)
}
```

Memory is bounded by the largest admitted facet family for one product. The manifest should declare a maximum facet count/bytes per product, with quarantine behavior for violations.

An even more efficient SQL may pre-aggregate facets, but the merge join is transparent, testable, and independent of database-specific JSON aggregation semantics.

9.3 Ordered source union

Each source cursor emits documents ordered by stable ID. Use a heap containing one head record per source:

```
heap = [head(products), head(categories), head(sql_docs)]
while heap not empty:
    record = pop minimum document ID
    reject duplicate document ID
    emit record
    push next record from the same source
```

Memory is `O(number of sources + one batch)`. The source priority and duplicate policy are compiled into identity.

9.4 Streaming corpus artifact

As documents are appended to the canonical relation:

- update the ordered semantic digest;
- write JSONL or a streamed JSON array to a temporary artifact;

- append exclusions/audit events to separate JSONL;
- record counts and bytes.

The artifact is sealed and renamed only after source completion and relation validation.

9.5 Two-pass furniture algorithm

A scalable exact approach:

Pass 0: role counts

Count admitted documents per role as a streaming fold.

Pass 1: emit per-document unique shingles

For each document:

1. tokenize into bounded spans;
2. generate normalized `q`-token shingles;
3. hash each shingle with BLAKE3/SHA-256;
4. deduplicate within the document using a bounded set, spilling if one document is extreme;
5. append `(role, hash, documentID)` to a partition chosen by hash prefix.

Pass 2: partition reduce

For each partition:

- sort/group by `(role, hash, documentID)` if needed;
- count distinct documents;
- compare against the role threshold;
- write qualifying `(role, hash, count)` rows.

Pass 3: rewrite documents

For each document:

- tokenize and generate hashes again;
- look up qualifying hashes using a prepared SQLite statement or bounded Bloom filter + exact database lookup;
- merge covered token runs;
- emit indexed text and audit spans.

Complexity is linear shingle generation plus external grouping I/O. Memory is bounded independently of corpus size.

An approximate heavy-hitter algorithm such as Count-Min Sketch can reduce the candidate set, but exact reviewability is valuable and the corpus size is moderate. Use approximation only as a prefilter, followed by exact counting.

9.6 Per-document chunking

Provide a cursor that emits one canonical/indexed document at a time. The chunker returns or emits chunks before the next document is read. The chunker's descriptor declares:

- maximum retained document/window bytes;
- overlap behavior;
- deterministic ID and byte-range rules;
- maximum chunks per document or a formula from document length;
- policy for malformed Markdown or oversized sections.

Chunk rows are appended immediately. Parent validation can be performed against the current document ID without a database lookup because execution is partitioned by document.

9.7 Representation construction

For each emitted chunk:

- create the raw representation;
- derive heading/breadcrumb text from the current document parser state;
- validate stable IDs and content digests;
- append representations immediately.

The current two-representation fanout is exact and should be represented as a component configuration, not hard-coded into the bundle builder.

9.8 Embedding cache and provider

Use a transactional cache keyed by:

```
embedding provider semantic type
+ model
+ dimensions
+ task-prefix contract
+ provider-normalization version
+ representation content digest
```

The cache result stores vector bytes, shape, finite-value validation status, creation metadata, and optional provider receipt. Suggested first backend: SQLite in WAL mode during the run, compacted/sealed afterward. A packfile/CAS can be added if write contention or database size becomes an issue.

Embedding batches should satisfy all limits:

```
records <= max_records
resident bytes <= max_batch_bytes
UTF-8 payload bytes <= provider_max_bytes
tokens <= provider_max_tokens
memory permit acquired
rate/call/token/cost permits acquired
```

The batcher should adapt to actual token estimates and observed vector/result size while never exceeding hard limits.

9.9 Index projections

After document/chunk/representation/embedding relations are sealed, construct projections under explicit reservations:

1. content projection;
2. lexical projection;
3. vector projection;
4. optional auxiliary metadata/authorization indexes.

Run them sequentially initially. Later, the planner can overlap them only when their combined reservation fits.

Each sink consumes an ordered cursor and returns:

```go
type SinkResult struct {
    SemanticInputDigest string
    BackendIdentity     json.RawMessage
```

```
    Artifact            ArtifactRef
    Counts              map[string]int64
    ObservedResources   ResourceObservation
}
```

9.10 Sealing and publication

A bundle is sealable only when:

- all required relations are sealed;
- counts and parent/foreign-key constraints pass;
- sink identities match planned semantic inputs;
- no temporary/provider work remains;
- artifact digests and sizes are recorded;
- the plan lock and source snapshot are copied into the run.

Publication writes a bundle manifest and receipt, fsyncs files/directories, and atomically creates the immutable generation. Activation is a later, separate operation.

9.11 Pseudocode for the target build

```
func Build(ctx context.Context, spec PipelineSpec) (PublishedBundle, error) {
    lock, err := compiler.Compile(ctx, spec)
    if err != nil { return PublishedBundle{}, err }

    run, err := runs.Create(lock)
    if err != nil { return PublishedBundle{}, err }

    snapshot, err := mysqlsource.OpenSnapshot(ctx, lock.Source)
    if err != nil { return fail(run, err) }
    defer snapshot.Close()

    relations, err := relationstore.Create(run.TempDir(), lock.Relations)
    if err != nil { return fail(run, err) }

    documents := planProductCategorySQLDocMerge(snapshot, lock)
    if err := executor.Drain(ctx, documents,
        relations.Documents().Appender(lock.Batches.Documents)); err != nil {
        return fail(run, err)
    }
    if err := relations.Documents().Seal(); err != nil { return fail(run, err) }

    indexedDocs := relations.Documents().Cursor()
    if lock.Furniture.Enabled {
        furniture, err := buildFurnitureRelation(ctx, relations.Documents(), lock)
        if err != nil { return fail(run, err) }
        indexedDocs = stripFurniture(ctx, relations.Documents(), furniture, lock)
    }

    if err := executor.RunPartitioned(ctx, indexedDocs,
        chunkRepresentEmbedAppend(relations, lock)); err != nil {
        return fail(run, err)
    }
    if err := relations.SealDerived(); err != nil { return fail(run, err) }

    results, err := sinks.BuildSequential(ctx, relations, lock.SinkSchedule)
    if err != nil { return fail(run, err) }
```

```
    candidate, err := bundle.Seal(run, lock, relations.Identities(), results)
    if err != nil { return fail(run, err) }
    if err := verifier.DeepVerify(ctx, candidate); err != nil { return reject(run, err) }
    return publisher.Publish(ctx, candidate)
}
```

The code is illustrative. The key is that product logic produces cursors/relations while the generic runtime owns admission, execution, sealing, and lifecycle.

10. Resource-bounded serving architecture

10.1 Compile the query path

The current query path is already conceptually a graph but is encoded as methods and setters. Compile it into a `ServingPlan`:

```
request
  -> validate/default
  -> comparison-intent expansion
  -> [lexical branch, vector branch]
  -> representation-to-chunk collapse
  -> metadata authorization per branch
  -> weighted RRF
  -> optional bounded rerank
  -> decomposition interleave/dedup
  -> final metadata authorization
  -> bounded content hydration
  -> evidence ledger + tool projection
```

The compiled plan should bind:

- query-transform identities;
- lexical/vector backend identities;
- over-fetch depth;
- fusion parameters;
- reranker pool and failure policy;
- authorization rules and trust boundaries;
- final result/evidence limits;
- request memory, time, provider, and token budgets.

The serving plan digest becomes part of every tool trace and evaluation artifact.

10.2 Candidate-budget algebra

Let:

- `Q` = number of retrieval queries after decomposition;
- `K` = requested final results;
- `a` = over-fetch factor;
- `K_l`, `K_v` = lexical and vector depths;
- `K_f` = fused candidate cap;
- `K_r` = reranker pool;
- `K_h` = final hydration cap.

The current path approximately uses:

$$K_l = K_v = aK, \quad a = 8$$

per retrieval query. A compiled plan should require explicit bounds:

$$Q \le Q_{max}$$

$$K_f \le \min(Q(K_l + K_v), K_{f,max})$$

$$K_r \le K_f, \quad K_h \le K$$

Memory estimates include hit structs, ID maps, metadata rows, reranker text, provider payload, and final hydrated text. This makes a change such as "increase default results from 5 to 8" analyzable as a resource change as well as a quality change.

10.3 Authorization as an information-flow constraint

CoinVault's authorization-before-hydration design should be made non-bypassable in the IR. Assign security labels to ports:

```
retrieval IDs/scores        label: identifiers
candidate metadata          label: scoped-metadata
source text                 label: scoped-content
external reranker input     sink: external-provider
model/tool output           label: user-visible
```

The compiler requires an `authorize` node before any edge from scoped content to an external provider or user-visible sink. A final authorization node remains defense in depth. Product code supplies the policy; the compiler enforces graph placement.

10.4 Startup verification tiers

Define explicit levels:

Level	Work	Typical use
`manifest`	Strict manifest parse, schema/version, expected bundle ID	inspection tools
`quick`	Manifest plus file digests/sizes, signed publication receipt, embedded backend identities	normal service startup
`deep`	Stream all semantic records and recompute logical/backend digests	promotion gate, periodic audit, forensic startup
`paranoid`	Deep verification plus independent backend query/property checks	release qualification

A bundle that passed deep verification before publication and is mounted read-only from a trusted artifact path normally needs only quick startup verification. This changes startup cost from corpus-size dependent to artifact-count dependent without weakening custody, provided receipts are signed or protected by an equivalent trusted channel.

10.5 Exact-vector backend: keep, but demote

The direct-blob optimization is correct. It changes exact search memory from repeated per-row vector allocation to a bounded query vector, one SQLite blob, and a top-K heap. It does not change the asymptotic cost:

$$T_{exact} = \Theta(Nd), \qquad IO_{exact} = \Theta(Nd \cdot sizeof(float32))$$

At the current corpus, one query scans about 668.6 MiB of vector payload. Multiple decomposed queries multiply that work. The backend is therefore appropriate for:

- small corpora;
- deterministic ground-truth evaluation;
- smoke and rollback bundles;
- low-query-rate environments;
- correctness comparison against approximate indexes.

Production-serving alternatives should implement the same `VectorIndex` descriptor and query contract:

```
type VectorIndex interface {
    SearchVector(ctx context.Context, q []float32, limit int) ([]Hit, error)
    Identity() VectorBackendIdentity
    Close() error
}
```

Candidate backends:

- HNSW with bounded construction parameters and optional mmap;
- IVF-flat or IVF-PQ;
- disk-oriented ANN;
- PostgreSQL/MySQL-compatible vector service where operationally justified;
- managed vector service behind an explicit external-effect adapter.

Approximate retrieval introduces recall and nondeterminism concerns. The bundle identity records index parameters and implementation version; evaluation compares ANN results to exact ground truth on a frozen query set; hard gates constrain recall degradation.

10.6 Query-time memory pool

Each request receives a child memory pool. Branches acquire permits for hit buffers, ID sets, metadata, reranker payloads, and hydrated text. The service has a global pool and admission queue so many simultaneous queries cannot each consume their maximum independently.

For request `r`:

$$M_r = M_{hits} + M_{metadata} + M_{rerank} + M_{hydration} + M_{response}$$

Global admission requires:

$$\sum_{r \in active} M_r + M_{service-base} \leq B_{service}$$

Use deadlines and queue limits to reject overload rather than allowing cgroup OOM.

10.7 Caches

Serving caches should be semantic and bounded:

- query embedding cache keyed by embedding contract and query bytes;
- lexical query result cache keyed by bundle and query-transform identity;
- vector candidate cache keyed by vector backend identity and query-vector digest;
- metadata/content page cache delegated to SQLite/OS where practical;
- reranker cache keyed by reranker identity, query, and candidate text digests.

Every cache needs an explicit byte capacity, eviction policy, and observability. Avoid unbounded maps hidden in services.

10.8 Hot reload and activation

Activation should update an immutable generation pointer, not mutate a live bundle. A service can support hot reload with a generation manager:

1. resolve active generation and quick-verify receipt;
2. open new bundle and warm bounded probes;
3. atomically swap a reference-counted service pointer;
4. drain old requests;
5. close old bundle;
6. retain rollback generation.

The activation operation is effectful and should require a promotion receipt. The pipeline kernel prepares the plan; the deployment owner performs the swap.

10.9 Tool boundary

`knowledge_search` should be generated from the compiled serving plan:

- description and input schema;
- default/max result limits;
- allowed comparison intents;
- evidence ledger limits;
- bundle and plan identities;
- source-role/scope policy;
- output schema and trace fields.

This removes drift between service defaults, tool descriptions, evaluation assumptions, and optimization candidate assets.

11. DSL, Go API, and plugin model

11.1 Why a DSL

A DSL is useful when it is a serializable input to a compiler, not a replacement programming language. It should expose composition, identity, resource policy, and effects while keeping complex algorithms in versioned components.

Recommended authoring formats:

- YAML for approachable manifests;
- CUE or JSON Schema for validation/defaults;
- canonical JSON for compiled identity;
- CEL-like expressions only for bounded predicates/policies.

Do not embed arbitrary Go templates or shell snippets as normal operators.

11.2 Example CoinVault build specification

```yaml
api_version: rag-pipeline/v1
name: coinvault-full-hybrid

budget:
  memory: 768MiB
  temp_disk: 24GiB
  output_disk: 12GiB
  provider:
    embedding_calls: 120000
    embedding_tokens: 200000000
  margin: 20%

inputs:
  mysql:
    component: mysql.consistent_snapshot@v1
    config:
      profile: coinvault-readonly
      isolation: repeatable-read
  sql_docs:
    component: coinvault.sql_docs@v1
    config:
      asset: asset://coinvault/sql-docs/v3

nodes:
  product_rows:
    component: coinvault.mysql_products@v2
    inputs: {snapshot: mysql}
    resources:
      max_record_bytes: 2MiB

  facet_rows:
    component: coinvault.mysql_product_facets@v2
    inputs: {snapshot: mysql}

  products:
    component: relational.merge_join@v1
    inputs:
      left: product_rows
      right: facet_rows
    config:
      key: product_id
      right_group_limit_records: 256
      right_group_limit_bytes: 256KiB

  product_documents:
    component: coinvault.normalize_product@v3
    inputs: {rows: products}
    config:
      min_description_runes: 200
      live_fact_policy: exclude-price-cost-inventory

  category_documents:
    component: coinvault.mysql_categories@v2
    inputs: {snapshot: mysql}
    config:
```

```
    min_description_runes: 120

schema_documents:
  component: coinvault.normalize_sql_docs@v2
  inputs: {docs: sql_docs}

documents:
  component: relational.merge_union@v1
  inputs:
    products: product_documents
    categories: category_documents
    sql_docs: schema_documents
  config:
    key: id
    duplicate_policy: reject

document_relation:
  component: store.relation_sink@v1
  inputs: {records: documents}
  config:
    relation: document
    checkpoint_every_records: 1000
    checkpoint_every_bytes: 32MiB

furniture:
  component: text.external_furniture@v1
  inputs: {documents: document_relation}
  config:
    group_by: metadata.source_role
    shingle_tokens: 12
    min_document_fraction: 0.05
    min_span_tokens: 20
    partitions: 64

indexed_documents:
  component: text.apply_furniture@v1
  inputs:
    documents: document_relation
    furniture: furniture

chunks:
  component: rag.markdown_heading_chunk@v2
  inputs: {documents: indexed_documents}
  config:
    max_section_runes: 1600
    min_section_runes: 200
    overlap_runes: 120

representations:
  component: rag.represent@v1
  inputs: {chunks: chunks}
  config:
    kinds: [raw-v1, breadcrumb-v1]

embeddings:
  component: embedding.cached@v2
  inputs: {representations: representations}
```

```
    config:
      profile: openai-small
      task_prefix: raw-v1
      batch:
        max_records: 64
        max_bytes: 1MiB
        max_tokens: 8000
      cache: relation://embedding_cache

  content_index:
    component: rag.content.sqlite@v2
    inputs:
      documents: indexed_documents
      chunks: chunks

  lexical_index:
    component: rag.lexical.bleve@v2
    inputs:
      documents: indexed_documents
      chunks: chunks
      representations: representations
    resources:
      reservation: calibrated://bleve/coinvault-full-v1

  vector_index:
    component: rag.vector.hnsw@v1
    inputs:
      representations: representations
      embeddings: embeddings

outputs:
  bundle:
    component: rag.bundle.seal@v3
    inputs:
      documents: document_relation
      chunks: chunks
      representations: representations
      embeddings: embeddings
      content: content_index
      lexical: lexical_index
      vector: vector_index
    policy:
      deep_verify_before_publish: true
      compatibility_exports: [corpus-jsonl]
```

The compiler may decide to materialize `chunks` and `representations` because multiple sinks consume them. The author does not manually wire temporary SQLite tables.

The same spec may carry optional structural declarations without exposing raw `Sigma/Delta/Pi` notation:

```
schemas:
  mysql_catalog: schema://coinvault/mysql-catalog/v4
  canonical_document: schema://rag/document/v2
  chunk_graph: schema://rag/chunk/v2

migrations:
  catalog_to_document:
```

```
    source: mysql_catalog
    target: canonical_document
    structural_map: migration://coinvault/catalog-document/v3
    value_program: coinvault.normalize_catalog@v3
    laws:
      - law://coinvault/live-facts-not-indexed/v1
      - law://coinvault/document-id-stability/v2
```

This is metadata for type checking, lineage, compatibility, and law tests. The physical compiler still emits ordered scans, merge joins, bounded transforms, and relation sinks.

11.3 `explain` output

Before execution:

```
$ ragpipe explain coinvault-full-hybrid.yaml

Logical digest: lp-sha256:...
Resolved registry: registry-sha256:...
Source consistency: repeatable-read snapshot
Estimated shape:
  documents          20,400  (p95 bytes 18 KiB)
  chunks             59,000
  representations   118,000
  embedding bytes   725 MiB

Materializations:
  document relation       required: furniture barrier + replay
  furniture relation      required: global aggregate
  chunks relation          required: fanout to content/lexical
  representations relation required: lexical/vector/verification fanout
  embeddings relation      required: paid side effect + vector build replay

Peak plan:
  stage lexical-build
  process/base             92 MiB
  Bleve reservation       430 MiB
  edge/batch/workspace     61 MiB
  margin                  140 MiB
  estimated peak          723 MiB / 768 MiB

Warnings:
  - memory margin below preferred 25%
  - exact-vector backend would read ~692 MiB per vector query
  - furniture pass estimated 14 GiB temporary shingle I/O

Plan admitted: yes
```

This output makes resource tradeoffs visible before an ECS task is launched.

11.4 Go builder API

A Go API should compile to the same IR:

```
plan := pipeline.New("coinvault-full-hybrid").
    Budget(resources.Memory(768<<20)).
    Input("mysql", mysql.ConsistentSnapshot(mysqlConfig)).
    Node("products", coinvault.Products().From("mysql")).
```

```
    Node("facets", coinvault.Facets().From("mysql")).
    Node("joined", relational.MergeJoin("products", "facets", "product_id")).
    Node("documents", coinvault.NormalizeProducts().From("joined")).
    // ...
    Output("bundle", ragbundle.Seal().From(...))

lock, err := compiler.Compile(ctx, plan.IR())
```

The Go builder improves refactoring and static help but does not bypass compilation.

11.5 Plugin tiers

Tier 1: in-process trusted Go components

Best for high-throughput, memory-sensitive core operators. Registered by static linking. No Go `plugin` package.

Tier 2: out-of-process typed plugins

Use gRPC/Connect/Protobuf or Arrow Flight/IPC for Python, Rust, or separately deployed services. The descriptor declares process memory limits, sandbox policy, transport batch bytes, and semantic version. Data crosses an explicit trust/resource boundary.

Tier 3: digest-pinned scripts

For experimentation and migration only. A script node declares:

- immutable image/script digest;
- input/output schema;
- maximum bytes/records;
- timeout/memory/cpu limits;
- deterministic claim;
- network policy;
- cacheability and effects.

Script output is strictly validated before entering trusted relations. It should never be an implicit shell command embedded in YAML.

11.6 Registry and lockfile

A registry entry includes:

```yaml
component: rag.lexical.bleve
version: 2.1.0
implementation_digest: sha256:...
config_schema: schema://rag/bleve/v2
input_ports: ...
output_ports: ...
semantic_identity_fields: ...
resource_model: ...
compatibility:
  semantic_output: lexical-records/v1
```

Compilation resolves ranges to exact versions and writes them to the lockfile. Production execution never resolves `latest`.

11.7 Package layout

A possible Go repository layout:

```
pipeline/
  ir/              canonical logical and physical plan types
  schema/          record/port schemas and compatibility
  registry/        descriptors and version resolution
  compiler/        validation, analysis, planning, lockfiles
  resource/        byte permits, rates, budgets, observations
  executor/        cursors, emitters, stages, cancellation
  relation/        relation store interfaces
  relation/sqlite/ first durable implementation
  artifact/        manifests, receipts, publication states
  runstore/        events, checkpoints, resume
  plugin/          process protocol and sandbox adapters
  explain/         plan and resource reports

ragkit/
  rag/...          domain types and component implementations
  components/...   descriptors/adapters for pipeline registry

coinvault/
  internal/knowledgepipeline/
    sources/       MySQL and SQL-doc product components
    transforms/    normalization, furniture policy
    serving/       product policy and tool projection
    registry.go    product component registration
  cmd/...          thin plan/run/verify/publish/activate commands
```

RagOpt can consume the same `Plan` and apply typed patches to component config or topology.

12. Engineering and operational workflow

12.1 The desired command lifecycle

Replace many incident-specific top-level flows with a small consistent interface:

```
ragpipe validate   spec.yaml
ragpipe plan       spec.yaml --lock plan.lock.json
ragpipe explain    plan.lock.json
ragpipe run        plan.lock.json --run-root ...
ragpipe resume     run-dir
ragpipe inspect    run-dir|artifact
ragpipe verify     artifact --level deep
ragpipe publish    verified-artifact --destination ...
ragpipe promote    published-receipt --environment dev
ragpipe activate   promotion-plan --environment dev
ragpipe rollback   environment --generation previous
```

CoinVault may wrap these commands with product defaults, but the underlying lifecycle and receipts remain generic.

12.2 Plan-first review

Every production build change should review:

- logical-plan diff;
- resolved component/version diff;
- semantic identity impact;
- source snapshot/query changes;
- resource-plan diff;

- materialization and spill changes;
- provider budget/cost diff;
- security-flow diff;
- expected artifact migration;
- evaluation and rollback requirements.

A code diff alone is not an adequate review representation.

12.3 Event-sourced run record

Runs emit append-only events:

```
run.created
plan.admitted
source.snapshot.opened
relation.batch.committed
operator.checkpoint
provider.call.reserved
provider.call.completed
relation.sealed
sink.started
sink.completed
artifact.sealed
verification.completed
publication.completed
run.completed|failed
```

Events carry stable low-cardinality fields and references to detailed artifacts. A materialized run summary can be rebuilt from the log.

This subsumes much of the current state-inspector and telemetry glue while retaining dedicated diagnostic views.

12.4 Observability

For each operator/stage record:

- input/output records and canonical bytes;
- estimated versus actual resident bytes;
- root/stage/operator permit high-water marks;
- heap, RSS, cgroup, GC, and native reservation;
- wall, CPU, blocked, provider, and I/O time;
- sequential/random bytes read/written;
- spill bytes and partitions;
- queue depth/bytes and residence time;
- cache hits/misses/writes;
- provider calls/tokens/cost;
- checkpoint/retry counts.

Keep high-cardinality IDs in run artifacts, not metric dimensions. EMF/OpenTelemetry can export summaries.

12.5 Testing hierarchy

Operator tests

- schema and invariant validation;
- deterministic output and identity;
- maximum record/batch enforcement;

- cancellation and permit release;
- spill-path parity with in-memory reference;
- failure injection.

Plan/compiler tests

- invalid graph/order/security/effect rejection;
- deterministic lockfile;
- memory proof arithmetic;
- materialization placement;
- physical-plan semantic parity.
- schema-migration composition and commuting-diagram tests on generated finite instances;
- direct source-to-current-schema parity against staged schema-upgrade paths.

Synthetic shape tests

Generate configurable distributions, not only counts:

- document length percentiles and outliers;
- facet-family sizes;
- chunk expansion;
- representation lengths;
- cache hit/miss patterns;
- vector dimensions;
- native backend calibration.

Run under hard memory and disk limits. The existing production-shape RagKit test is a useful seed but should include the eager front-half algorithms it currently bypasses.

Golden full-corpus tests

On approved snapshots, compare:

- document/chunk/representation identities;
- bundle semantic ID;
- lexical/vector retrieval order on frozen queries;
- authorization behavior;
- tool output/evidence identity;
- memory and duration envelopes.

Failure/recovery tests

Inject failure:

- after every committed batch;
- during provider call and after provider success/before commit;
- during each sink;
- before/after sealing;
- during fsync/rename/upload;
- during activation.

Resume must either continue exactly or fail closed with a clear custody reason.

12.6 CI gates

Suggested gates:

1. `go test`, race, lint, generation.
2. Descriptor/schema compatibility checks.

3. Plan compilation for committed production specs.
4. No-unbounded-operator rule: every global operator has spill or finite ceiling.
5. No-hidden-materialization rule: selected packages cannot call whole-corpus helpers without an explicit relation/barrier annotation.
6. Deterministic lockfile and bundle-semantic-ID tests.
7. Small cgroup memory test on synthetic production shape.
8. Security-flow compile checks.
9. Artifact open/quick/deep verification parity.
10. Query correctness and bounded request-memory tests.

12.7 Capacity engineering

Capacity should be derived from the physical plan and calibrated evidence:

```
requested container memory
  = admitted plan peak
  + OS/runtime baseline variance
  + native-backend uncertainty
  + page-cache policy allowance
  + operational margin
```

Maintain separate resource classes for:

- source snapshot/ingest;
- full index build;
- deep verifier;
- exporter/packager;
- serving;
- evaluation.

Do not choose one task size for all workloads.

12.8 Artifact workflow

Every artifact has one manifest:

```go
type ArtifactManifest struct {
    APIVersion      string
    ArtifactID      string
    Role            string
    SemanticDigest string
    PhysicalDigest string
    SizeBytes       int64
    ProducerPlan    string
    RunID           string
    Inputs          []ArtifactRef
    State           string // temporary, sealed, verified, published, active
    Verification    []VerificationReceipt
    Locations       []Location
}
```

Cache export, bundle export, seed, S3 receipt, EFS state, and promotion can then use generic artifact operations with product-specific policies.

12.9 Work organization

The OOM ticket mixed investigation, infrastructure, product changes, generic RagKit changes, serving changes, and deployment handoff. For future work, organize around stable vertical capabilities:

- **Kernel capability ticket**: e.g., external aggregate operator with resource contract.
- **Product adapter ticket**: CoinVault furniture component using that operator.
- **Proof ticket**: full-corpus bounded-memory and semantic-parity evidence.
- **Deployment ticket**: publish/activate the proven artifact.

Each ticket should point to one plan/receipt lineage. This keeps diaries useful without making them the only integration architecture.

13. Migration strategy

13.1 Principles

- Preserve current semantic bundle and serving behavior while changing execution.
- Do not attempt a universal distributed engine first.
- Make resource contracts executable before adding broad plugin flexibility.
- Keep the current `BuildStream`, verifier, content store, and exact backend as compatibility adapters.
- Move one global materialization at a time.
- Require full-corpus identity/retrieval parity for pure physical changes.

13.2 Phase 0: freeze the baseline

Deliverables:

- exact source branch/commit and RagKit pin;
- one approved source snapshot or exported canonical corpus;
- current schema-v2 bundle and deep-verification receipt;
- frozen retrieval/evaluation suite;
- measured build/startup/query resource traces;
- explicit current artifact inventory.

Resolve the archive/worktree reproducibility issue. A baseline cannot depend on source recovered from traces.

13.3 Phase 1: introduce IR, descriptors, and resource contracts

Wrap existing components without changing behavior:

- product/category/SQL-doc source descriptors;
- eager normalize/furniture/chunk/representation adapter nodes;
- current `BuildStream` as one sink node;
- current `indexbundle.Open` and service as one serving node;
- compile committed specs into lockfiles;
- produce `explain` with initially conservative/manual resource models.
- register the current MySQL, document, chunk, representation, embedding, bundle, and serving-view schemas;
- declare source-to-document and bundle-to-view structural maps plus parity laws, while keeping existing Go functions as value programs.

The first value is visibility and drift elimination, not lower memory.

13.4 Phase 2: stream MySQL ingestion into canonical documents

Implement:

- consistent snapshot component;
- product/facet merge join;
- category and SQL-doc cursors;

- ordered merge union;
- streaming corpus artifact;
- canonical document relation with checkpoints.

Remove the corpus-sized facet map, document slices, global sort, and whole-file JSON marshal from the production path. Keep an eager reference implementation for tests.

Acceptance:

- identical admitted/excluded documents and ordered digests;
- bounded memory under adversarial facet/document sizes;
- resume after committed document batches;
- source snapshot receipt.

13.5 Phase 3: per-document chunk and representation pipeline

Implement partitioned chunking and representation emission. Append to relations immediately. Remove global `[]Chunk`, raw-representation, breadcrumb-representation, and composed-representation slices.

Acceptance:

- identical chunk and representation IDs/digests/order;
- bounded memory by maximum admitted document plus batches;
- exact bundle semantic parity for no-furniture manifests.

This is the highest-value first vertical slice because it establishes true end-to-end boundedness for the simplest corpus.

13.6 Phase 4: transactional embedding relation/cache

Replace file-per-item cache in the main path with a transactional cache/relation adapter while preserving the old cache importer/exporter.

Acceptance:

- provider-free replay and fail-closed misses;
- exact vector parity;
- atomic cache/output commits;
- bounded token/byte batches;
- materially reduced inode/export overhead.

13.7 Phase 5: external furniture

Implement the two-pass external frequency and rewrite plan. Keep the current in-memory implementation as a small-corpus oracle.

Acceptance:

- exact stripped-text/report parity on fixtures and approved corpus;
- bounded memory independent of corpus size;
- temporary-disk estimate and cleanup proof;
- plan can reuse furniture relation when source/config identity is unchanged.

13.8 Phase 6: consolidate relation and bundle storage

Adopt Option B initially: retain `relations.sqlite` plus specialized indexes, and make JSON compatibility artifacts optional. Refactor `BuildStream` around relation cursors rather than its hard-coded stager API.

Acceptance:

- no unnecessary corpus-sized duplicate by default;
- artifacts remain inspectable and rebuildable;
- quick/deep verification receipts;
- atomic publication and existing serving compatibility.

13.9 Phase 7: compile serving plans

Represent the current query graph and tool contract in the IR. Add request memory pools, explicit candidate budgets, verification tiers, and generated tool metadata.

Acceptance:

- identical retrieval/tool behavior on frozen cases;
- bounded concurrent-request memory;
- security-flow compile checks;
- quick startup does not scan the full corpus;
- deep audit remains available and required before promotion.

13.10 Phase 8: pluggable production vector backend

Add an ANN implementation behind the vector interface. Retain exact SQLite for ground truth and rollback.

Acceptance:

- recall/quality gates against exact search;
- latency, I/O, memory, build-time, and disk measurements;
- deterministic/sealed backend identity;
- resource model and failure/rollback path.

13.11 Phase 9: connect to RagOpt

Expose logical plan nodes and resource policy as typed patch targets. RagOpt can then optimize:

- chunker parameters;
- representation kinds;
- embedding contracts;
- lexical/vector backends and parameters;
- fusion/reranker budgets;
- materialization/physical plans for cost, provided semantic parity constraints apply;
- memory/latency/cost tradeoffs on a Pareto frontier.

A physical-plan candidate that claims semantic equivalence must pass identity and retrieval parity gates before quality optimization.

14. Concrete recommendations for the current codebase

14.1 Keep

- `indexbundle.BuildStream` semantics: fail-closed admission, ordered sealing, atomic publication.
- streaming verifier and paged backend inspection.
- schema-v2 `content.Store` and authorization-before-hydration.
- embedding profile identity checks and task-prefix identity.
- cache-only provider rejection for recovery proof.
- direct vector-blob scoring for the exact backend.
- separate indexer/serving capacities and read-only serving mount.
- memory/RSS/cgroup telemetry, but integrate it into generic stage observations.

- bundle/cache export receipts and immutable-generation discipline.

14.2 Refactor

Current area	Refactor target
`cmd/coinvault/cmds/knowledge.go`	Thin commands over generic plan/run/artifact/eval APIs; split product-specific eval/judge commands if retained
`knowledgebuild.Build`	CoinVault plan factory plus source/transform component registrations
`LoadProductDocuments`	ordered source cursor with merge-joined facets
`LoadCategoryDocuments`	ordered source cursor
`SQLDocDocuments`	ordered source cursor or sealed asset relation
`StripFurniture`	external two-pass operator with eager oracle implementation
`chunking.Apply` whole-corpus call	per-document partition operator
representation slice composition	per-chunk fanout operator
per-file embedding cache	transactional cache relation; compatibility importer/exporter
`indexbundle.Stager`	relation-store sink adapter behind generic ports
startup `Open -> Verify(deep)`	quick verification receipt + optional deep mode
`Service` setters	compiled serving-plan configuration
S3/EFS export/state commands	generic artifact store and manifest operations

14.3 Remove from the production path after parity

- corpus-sized `[]rag.Document`, `[]rag.Chunk`, and `[]rag.Representation` ownership in the builder;
- `json.MarshalIndent` over whole corpus collections;
- global product facet map;
- disposable staging data that is fully duplicated into several final authorities without an explicit reason;
- record-count-only batch contracts;
- implicit environment-driven serving mechanisms not captured in a serving lockfile;
- deep corpus scan on every normal startup.

14.4 Do not do

Do not build a generic categorical database runtime first

Use schema categories, migrations, and composition laws in the logical IR and tests. Compile them to ordinary relational/external operators. A production dependency on a generic Kan-extension evaluator would make resource behavior harder to inspect and would not address text transforms, provider effects, or index construction.

Do not build a generic `Stream[T]` package and stop there

Channels/cursors alone do not express global barriers, memory ownership, spill, native reservations, semantic identity, or artifacts.

Do not introduce distributed execution first

The corpus fits comfortably on one machine with external-memory algorithms. Distribution adds partition determinism, network backpressure, failure coordination, and artifact-shuffle complexity

before the local model is sound.

Do not retain every compatibility artifact by default

Generate human-readable JSON exports on demand or as policy-selected outputs. Authority should be explicit.

Do not auto-parallelize index sinks

Bleve is the measured peak. Sequential construction is predictable and often fast enough. Parallelism should be admitted by the resource planner, not assumed.

Do not replace exact search without an evaluation contract

Keep it as the ground-truth backend. Approximate search is a semantic/quality change unless proven within declared recall constraints.

Do not let scripts hide resource behavior

Every escape hatch must declare and enforce memory, CPU, disk, network, schema, and identity boundaries.

15. First vertical slice in detail

The recommended first delivery is a **no-furniture, product-only, end-to-end bounded build**. It proves the kernel without solving the hardest global transform simultaneously.

15.1 Scope

- products and facets from one consistent MySQL snapshot;
- existing product normalization contract;
- heading chunker;
- raw + breadcrumb representations;
- existing embedding contract/cache adapter;
- existing content/Bleve/exact-vector outputs through relation cursors;
- same schema-v2 serving behavior;
- same bundle semantic identity if feasible, or an explicitly mapped schema-v3 identity with retrieval parity.

15.2 Resource target

Choose a hard target such as 512 MiB or 768 MiB after measuring the native Bleve envelope. The target must be low enough to catch hidden materialization but high enough to include the calibrated lexical sink plus margin.

15.3 Proof obligations

- no corpus-sized Go slice/map in the source-to-relation path;
- all batches have record and byte ceilings;
- one-record pathological limit tests;
- product/facet merge parity;
- ordered document/chunk/representation/vector digest parity;
- provider-free cache replay;
- fail/restart after each checkpoint;
- build under hard cgroup limit;

- quick/deep verification;
- frozen-query retrieval and authorization parity;
- artifact publication and rollback.

15.4 Why this slice

It replaces the most important eager boundaries while reusing the strongest current work. Categories and SQL docs are straightforward ordered sources after the kernel exists. Furniture is then an isolated external aggregate rather than entangled with initial runtime design.

16. Success criteria

A successful foundation should meet these measurable outcomes.

16.1 Correctness and identity

- deterministic logical plan and lockfile;
- deterministic ordered relations;
- stable semantic identities independent of physical batch/workers;
- exact parent/foreign-key and count validation;
- no activation without verified publication receipt;
- query/evidence traces include bundle and serving-plan identities.

16.2 Resource safety

- full source-to-bundle build admitted under a declared memory/disk/provider budget;
- no operator with corpus-sized retained state unless a finite ceiling is proven;
- actual high-water remains within the admitted envelope and margin;
- bounded verification and serving startup;
- bounded concurrent request memory;
- temporary files cleaned or recoverably classified after failure.

16.3 Simplicity

- one canonical relation/artifact vocabulary;
- one lifecycle for build, verify, publish, and activate;
- thin product CLI adapters;
- fewer one-off export/state/seed mechanisms;
- `explain` shows every barrier, materialization, spill, and high-water stage;
- operators are independently testable and reusable.

16.4 Performance

- sequential source scans and merge joins rather than global maps/sorts;
- bounded provider batches with high cache reuse;
- minimized disk amplification;
- calibrated native sink scheduling;
- production vector query backend selected by measured quality/latency/resource tradeoff;
- startup quick verification proportional to artifact count, not corpus rows.

16.5 Engineering workflow

- committed specs compile in CI;
- resource-plan changes are reviewable separately from semantic changes;

- full-corpus proof runs produce immutable receipts;
- failure injection and resume are routine tests;
- RagOpt can patch registered plan nodes without adding product command branches.

17. Risks and tradeoffs

17.1 More formalism

Descriptors, schemas, and compiler passes add up-front work. The mitigation is to keep the initial operator set small and wrap existing functions. The current incident already paid a larger informal complexity cost across many packages and operational tools.

17.2 SQLite contention and disk amplification

SQLite is pragmatic but can bottleneck concurrent writers and create write amplification. Use one writer per relation, bounded transactions, sequential stages, and WAL only where necessary. Preserve the store interface so a different backend can replace it later.

17.3 Resource estimates can be wrong

Native libraries and workloads vary. Use conservative margins, actual runtime enforcement, percentile calibration, and hard cgroup tests. The planner should expose confidence and unknown terms rather than claim false precision.

17.4 External algorithms may be slower

A bounded two-pass algorithm can perform more I/O than an in-memory algorithm. The planner may choose the in-memory physical implementation when the corpus has a proven upper bound and budget permits. Semantic configuration remains the same.

17.5 Over-generalization

A universal data platform would be a mistake. The kernel should support the operators CoinVault and RagKit actually need: ordered streams, partitioned transforms, external sort/group/join, side-effect batches, relations, sinks, and artifact lifecycle. Add features only after two real consumers require them.

17.6 Plan/implementation drift

Descriptors can lie. CI and runtime should verify that implementations consume permits, enforce declared limits, emit expected schemas, and report actual high-water. Trusted components require code review; out-of-process plugins require sandbox enforcement.

17.7 Identity migration

Consolidating storage or changing canonical serialization may change bundle IDs despite identical retrieval semantics. Define migration mappings and separate logical identity from physical artifact identity before changing formats. Avoid forcing byte-identical SQLite files as the proof criterion.

17.8 Category-theory overreach

A categorical vocabulary can improve schema composition and law-driven testing while making the project less approachable if it becomes the public programming model. Keep user-facing concepts concrete: schema, relation, mapping, join, view, key, resource budget, and artifact. Reserve

`Sigma/Delta/Pi`, adjunctions, and naturality for documentation, compiler internals, and advanced tooling. The success criterion is simpler, safer pipelines - not the amount of formal terminology in the codebase.

18. Final recommendation

The immediate memory work should be merged or preserved as the compatibility baseline, but further effort should stop extending the current split architecture one local mechanism at a time.

The next foundation should be:

> A single-machine, resource-bounded, typed dataflow/build kernel with a canonical relation store, explicit external-memory operators, byte-credit backpressure, compiled semantic and physical plans, immutable artifact lifecycle, and product/plugin boundaries.

Its logical layer should include versioned schemas and composable schema migrations, borrowing Spivak/Fong's functorial laws for structural integration and view equivalence. Its runtime should remain an explicit relational/external-memory engine with measurable resource contracts.

This kernel makes the entire CoinVault path legible:

```
consistent MySQL snapshot
  -> ordered source relations
  -> bounded normalization
  -> explicit global transformations
  -> per-document chunk/representation transducers
  -> transactional embedding/cache relation
  -> sequential resource-reserved index projections
  -> sealed and deeply verified immutable bundle
  -> signed publication receipt
  -> quick-verified, request-bounded serving plan
  -> tool/evidence outputs with exact lineage
```

It also explains which current mechanisms belong where:

- RagKit domain types and indexes remain implementations in the data plane.
- `BuildStream` becomes a compatibility physical backend or relation adapter.
- the content store remains the bounded serving authority.
- current telemetry becomes generic execution observation.
- cache/export/seed/state tooling becomes generic artifact custody.
- CoinVault retains source semantics, access policy, and tool behavior.
- RagOpt operates on the same canonical plans and resource-aware patch space.

The decisive design test is simple: a reviewer should be able to run `explain` and answer, before execution:

1. What are the exact source and semantic inputs?
2. Which operators are streaming, partitioned, global, or side-effecting?
3. Where are the materialization and recovery boundaries?
4. What is the conservative memory/disk/provider bound for every stage?
5. Which artifacts are authoritative and which are derivable views?
6. What exact plan will serving execute, with what request limits and trust boundaries?
7. What evidence is required before publication and activation?

Today those answers exist, but across source files, diaries, task definitions, telemetry, manifests, exporter receipts, and deployment configuration. The proposed kernel turns them into one executable design.

Appendix A. Current full-shape arithmetic

A.1 Vector payload

Given:

```
representations N = 114,106
dimensions      d = 1,536
float32 bytes     = 4
```

raw vector bytes are:

$$Nd4 = 701{,}067{,}264 \text{ bytes}$$

which is:

```
668.59 MiB
0.653 GiB
175,266,816 float components
```

This excludes:

- `rag.Vector` struct and slice headers;
- representation IDs and model strings;
- per-batch/provider allocations;
- cache objects;
- SQLite input blobs/pages;
- index build state;
- Go heap fragmentation and GC headroom.

A.2 Eager relation multiplicity

At the measured counts, the builder held or produced:

```
19,977 documents
57,053 chunks
114,106 representations
114,106 vectors
```

The representation count is exactly twice the chunk count, consistent with raw plus breadcrumb representations. Any memory model should make this fanout explicit rather than infer it after allocation.

A.3 Exact-query data movement

Each vector query scans at least the raw 668.59 MiB vector payload. With `Q` decomposed retrieval queries:

$$IO_{vectors} \gtrsim Q \times 668.59 \text{ MiB}$$

This is a lower bound before SQLite and filesystem overhead. Direct blob scoring fixes allocations, not data movement.

A.4 Observed checkpoints from the OOM work

Observation	Value
Cache files	113,719

Observation	Value
Cache archive/data size	about 2.22 GB
Cache hits before fail-closed miss in reproduction	56,000
RSS near half-vector accumulation	about 1.36 GiB
Synthetic streamed production shape	19,977 / 57,053 / 114,106 x 1,536
Synthetic maximum RSS	270,336 KiB
Synthetic duration	81.71 s
Exact full rebuild provider requests	0 after cache reconciliation
Exact full rebuild peak RSS	835,891,200 bytes
Serving peak under 2 GiB limit	252,141,568 bytes
Serving peak under 4 GiB limit	249,868,288 bytes

Measurements are taken from the project ticket/diaries and should be preserved as historical evidence, not treated as universal performance guarantees.

Appendix B. Current source-to-serving inventory

Concern	Current implementation area
Build command/orchestration	`cmd/coinvault/cmds/knowledge.go`
Source manifest	`internal/knowledgebuild/manifest.go`
MySQL/SQL-doc connectors	`internal/knowledgebuild/connectors.go`
Furniture algorithm	`internal/knowledgebuild/furniture.go`
Embedding runtime/prefix	`internal/knowledgebuild/embed.go`
Memory telemetry	`internal/knowledgebuild/memory.go`, telemetry files
Bundle staging/build	RagKit `rag/indexbundle/build_stream.go`, `staging_kernel.go`
Content store	RagKit `rag/content/sqlite/index.go`
Bundle open/verification	RagKit `rag/indexbundle/open.go`, `verify*.go`
Lexical backend	RagKit `rag/lexical/bleve`
Exact vector backend	RagKit `rag/vector/sqliteexact`
Product search/fusion/rerank	`internal/knowledge/service.go`
Bounded authorization/hydration	`internal/knowledge/content_lookup.go`
Tool projection/evidence	`internal/knowledge/tool.go`
Runtime mechanism assembly	`internal/knowledge/runtime_config.go`
Server activation	`cmd/coinvault/cmds/serve.go`, `internal/webchat/server/server.go`
Artifact export/state/seed	`internal/knowledgebundleexport`, `knowledgecacheexport`, `knowledgestateinspect`, `knowledgeseed`
Deployment/capacity	external CoinVault Terraform/ECS runtime module and environment roots

Appendix C. Suggested minimal interfaces

```
type Cursor[T any] interface {
    Next(context.Context) (*Batch[T], error)
    Close() error
}
```

```
type Emitter[T any] interface {
    Emit(context.Context, *Batch[T]) error
}

type Operator[A, B any] interface {
    Descriptor() Descriptor
    Run(context.Context, Cursor[A], Emitter[B], Runtime) error
}

type PartitionOperator[A, B any] interface {
    Descriptor() Descriptor
    RunPartition(context.Context, PartitionKey, Cursor[A], Emitter[B], Runtime) error
}

type ExternalOperator[A, B any] interface {
    Descriptor() Descriptor
    PlanSpill(context.Context, Estimate, ResourceBudget) (SpillPlan, error)
    RunExternal(context.Context, Cursor[A], Emitter[B], SpillStore, Runtime) error
}

type Relation[T any] interface {
    Identity() RelationIdentity
    Cursor(context.Context, CursorOptions) (Cursor[T], error)
}

type RelationAppender[T any] interface {
    Append(context.Context, *Batch[T]) error
    Checkpoint(context.Context) (Checkpoint, error)
    Seal(context.Context) (RelationIdentity, error)
    Abort(context.Context, error) error
}

type ArtifactStore interface {
    Begin(context.Context, ArtifactPlan) (ArtifactWriter, error)
    Open(context.Context, ArtifactRef) (ArtifactReader, error)
    Publish(context.Context, SealedArtifact, Destination) (PublicationReceipt, error)
}
```

The first implementation can simplify these interfaces. The non-negotiable properties are byte ownership, inspectable descriptors, deterministic identity, and explicit materialization/effects.

Appendix D. Literature and conceptual references

1. Goetz Graefe. “Volcano - An Extensible and Parallel Query Evaluation System.” *IEEE Transactions on Knowledge and Data Engineering*, 6(1), 1994. DOI: 10.1109/69.273032.
2. Alok Aggarwal and Jeffrey S. Vitter. “The Input/Output Complexity of Sorting and Related Problems.” *Communications of the ACM*, 31(9), 1988. DOI: 10.1145/48529.48535.
3. Edward A. Lee and David G. Messerschmitt. “Synchronous Data Flow.” *Proceedings of the IEEE*, 75(9), 1987.
4. Shuvra S. Bhattacharyya and Edward A. Lee. “Memory Management for Synchronous Dataflow Programs.” University of California, Berkeley technical report, 1992.
5. Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. “Build Systems a la Carte.” *Proceedings of the ACM on Programming Languages*, ICFP, 2018. DOI: 10.1145/3236774.
6. John D. C. Little. Little’s law for stable queueing systems: average number in system equals arrival rate times average time in system.
7. Gilles Kahn. “The Semantics of a Simple Language for Parallel Programming.” IFIP Congress,

1974. Kahn process networks motivate deterministic communication over channels with blocking reads.
8. Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017. Relevant concepts include logs, materialized views, stream processing, storage/index separation, and derived data.
9. David I. Spivak. "Functorial Data Migration." arXiv:1009.1166, 2010. Schemas as categories, instances as set-valued functors, and the induced migration functors.
10. David I. Spivak and Ryan Wisnesky. "Relational Foundations for Functorial Data Migration." arXiv:1212.5303, 2012. Relational expressiveness, composition, and the `Sigma-Delta-Pi` adjoint triple.
11. Ryan Wisnesky, David I. Spivak, Patrick Schultz, and Eswaran Subrahmanian. "Functorial Data Migration: From Theory to Practice." arXiv:1502.05947, 2015. Practical FQL experience and the case for compiling convenient query syntax directly.
12. Brendan Fong and David I. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. arXiv:1803.05316, 2018. Wiring, composition, databases, adjunctions, and applied categorical design.

Appendix E. Code and ticket references

The most load-bearing implementation references are CoinVault's `internal/knowledgebuild/{build,connector` on `agent/profile-backed-embeddings`; `cmd/coinvault/cmds/knowledge.go`; `internal/knowledge/{service,con` RagKit's `rag/indexbundle/{build_stream,staging_kernel,open,types,verify,verify_stream}.go` and `rag/content/sqlite/index.go` on `agent/oom-build-observer`; exact-vector commit `5a4a5e1f38451d1e79a71` and CoinVault ticket `COINVAULT-INDEX-OOM-001`, especially its changelog, task ledger, bounded-memory serving design, investigation diary, and deployment handoff.